

Отчет о выполнении лабораторной работы №1
в модуле №1.

Выполнил: Соломонов Д.М.

Проверил: Менжулин С.А.

Выполнение лабораторной работы №1. Раздел № 1

Выполним вход на сервер, используя команду: **ssh username@ip -p port**, в нашем случае команда имеет следующий вид: **ssh eltex-pg2-v11@217.71.138.1 -p 44556**

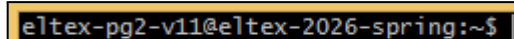


Рисунок 1 – Доступ на сервер

Далее запустим новую сессию утилиты screen, используя команду: **screen**

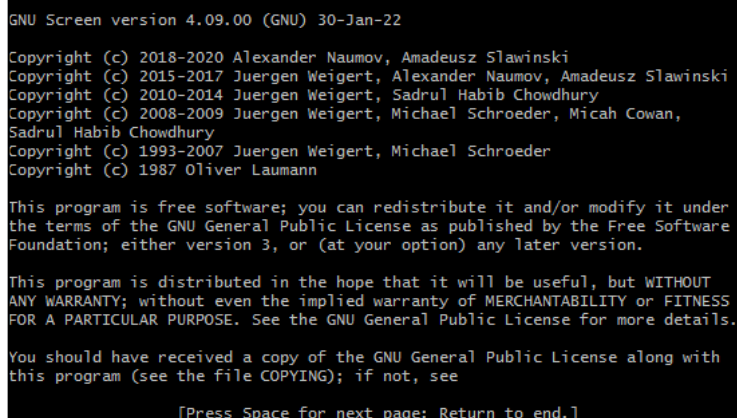


Рисунок 2 – Результат выполнения команды screen

Далее с помощью команды **w** посмотрим всех подключенных пользователей

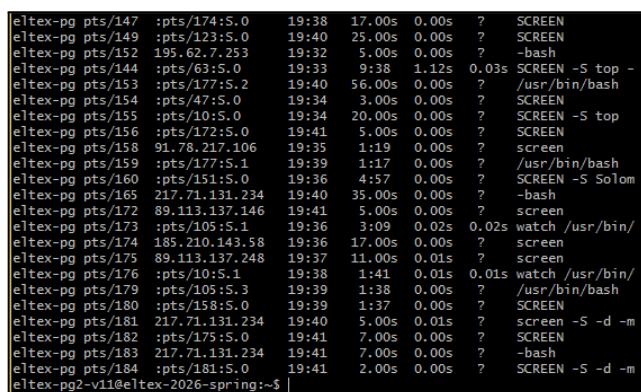


Рисунок 3 – Список пользователей

Далее покинем сессию, используя комбинацию клавиш **CTRL + A**, затем **D**:

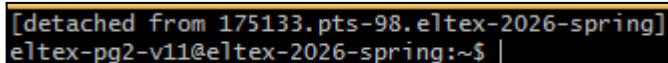


Рисунок 4 – Отключаемся от сессии

Далее запустим отсоединенную сессию, используя команду: **screen -r**

```
[detached from 175133.pts-98.eltex-2026-spring]
eltex-pg2-v11@eltex-2026-spring:~$ screen -r
There are several suitable screens on:
  175133.pts-98.eltex-2026-spring (03/05/2026 07:37:58 PM)      (Detache
d)
  172723.Solomon (03/05/2026 07:36:18 PM)      (Attached)
  167093.pts-98.eltex-2026-spring (03/05/2026 07:32:16 PM)      (Detache
d)
```

Рисунок 5 – Список отсоединенных сессий

Далее с помощью команды: **screen -r 175133.pts-98.eltex-2026-spring** подключимся к сессии

```
eltex-pg pts/147 :pts/174:S.0 19:38 17.00s 0.00s ? SCREEN
eltex-pg pts/149 :pts/123:S.0 19:40 25.00s 0.00s ? SCREEN
eltex-pg pts/152 195.62.7.253 19:32 5.00s 0.00s ? -bash
eltex-pg pts/144 :pts/63:S.0 19:33 9:38 1.12s 0.03s SCREEN -S top -
eltex-pg pts/153 :pts/177:S.2 19:40 56.00s 0.00s ? /usr/bin/bash
eltex-pg pts/154 :pts/47:S.0 19:34 3.00s 0.00s ? SCREEN
eltex-pg pts/155 :pts/10:S.0 19:34 20.00s 0.00s ? SCREEN -S top
eltex-pg pts/156 :pts/172:S.0 19:41 5.00s 0.00s ? SCREEN
eltex-pg pts/158 91.78.217.106 19:35 1:19 0.00s ? screen
eltex-pg pts/159 :pts/177:S.1 19:39 1:17 0.00s ? /usr/bin/bash
eltex-pg pts/160 :pts/151:S.0 19:36 4:57 0.00s ? SCREEN -S Solom
eltex-pg pts/165 217.71.131.234 19:40 35.00s 0.00s ? -bash
eltex-pg pts/172 89.113.137.146 19:41 5.00s 0.00s ? screen
eltex-pg pts/173 :pts/105:S.1 19:36 3:09 0.02s 0.02s watch /usr/bin/
eltex-pg pts/174 185.210.143.58 19:36 17.00s 0.00s ? screen
eltex-pg pts/175 89.113.137.248 19:37 11.00s 0.01s ? screen
eltex-pg pts/176 :pts/10:S.1 19:38 1:41 0.01s 0.01s watch /usr/bin/
eltex-pg pts/179 :pts/105:S.3 19:39 1:38 0.00s ? /usr/bin/bash
eltex-pg pts/180 :pts/158:S.0 19:39 1:37 0.00s ? SCREEN
eltex-pg pts/181 217.71.131.234 19:40 5.00s 0.01s ? screen -S -d -m
eltex-pg pts/182 :pts/175:S.0 19:41 7.00s 0.00s ? SCREEN
eltex-pg pts/183 217.71.131.234 19:41 7.00s 0.00s ? -bash
eltex-pg pts/184 :pts/181:S.0 19:41 2.00s 0.00s ? SCREEN -S -d -m
eltex-pg2-v11@eltex-2026-spring:~$ |
```

Рисунок 6 – Подключение к сессии

Далее запустим команду **top**:

```
top - 19:51:24 up 8:29, 72 users, load average: 0.16, 0.12, 0.09
Tasks: 620 total, 2 running, 613 sleeping, 5 stopped, 0 zombie
%Cpu(s): 1.6 us, 2.8 sy, 0.0 ni, 88.8 id, 6.3 wa, 0.0 hi, 0.2 si, 0.2 st
MiB Mem : 3914.7 total, 1861.9 free, 1130.2 used, 1167.6 buff/cache
MiB Swap: 976.0 total, 976.0 free, 0.0 used, 2784.5 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM    TIME+  COMMAND
 197346 eltex-pt  20   0   11752    524    3316 S   5.3   0.1   0:02.93 top
 179460 eltex-pt  20   0    8268    2668   1944 S   1.0   0.1   0:00.39 screen
  97823 eltex-pt  20   0   11640    5528   3388 S   0.7   0.1   0:20.99 top
 158862 eltex-pt  20   0   18012    7092   5220 S   0.7   0.2   0:00.17 sshd
  35426 eltex-pt  20   0    8444    2828   1948 S   0.3   0.1   0:00.66 screen
  37911 eltex-pt  20   0   11640    5504   3356 S   0.3   0.1   0:11.94 top
  43203 eltex-pt  20   0   11680    5308   3148 S   0.3   0.1   0:11.74 top
  74221 eltex-pt  20   0   11780    5460   3308 S   0.3   0.1   0:09.11 top
  79876 eltex-pt  20   0    6184    2944   2472 S   0.3   0.1   0:05.99 watch
  91120 eltex-pt  20   0    6184    2852   2432 S   0.3   0.1   0:05.57 watch
  92769 eltex-pt  20   0   11644    5572   3420 S   0.3   0.1   0:07.68 top
  96247 eltex-pt  20   0    8684    3032   1968 S   0.3   0.1   0:00.49 screen
  96881 eltex-pt  20   0   11620    5548   3400 S   0.3   0.1   0:07.39 top
 151494 eltex-pt  20   0   18012    7064   5196 S   0.3   0.2   0:00.04 sshd
 157446 eltex-pt  20   0    8936    3332   2012 S   0.3   0.1   0:00.07 screen
 175796 eltex-pt  20   0    6056    2860   2512 S   0.3   0.1   0:00.86 watch
 179659 eltex-pt  20   0   11804    5456   3304 S   0.3   0.1   0:01.39 top
```

Рисунок 7 – Результат выполнения команды **top**

Теперь переименуем текущую сессию используя комбинацию клавиш **CTRL + A**, после **shift + :** и назовем её **top** используя команду: **sessionname top**

Далее проверим результат изменения названия сессии используя команду: **screen -r**

```
175133.top      (03/05/2026 07:37:58 PM)      (Detached)
```

Рисунок 8 – Результат переименования сессии

Далее получим список созданных сессий, используя команду: **screen -r**

```
There are several suitable screens on:
 175133.top      (03/05/2026 07:37:58 PM)      (Detached)
 172723.Solomon  (03/05/2026 07:36:18 PM)      (Attached)
 167093.pts-98.eltex-2026-spring (03/05/2026 07:32:16 PM)      (Detached)
Type "screen [-d] -r [pid.]tty.host" to resume one of them.
eltex-pg2-v11@eltex-2026-spring:~$ |
```

Рисунок 9 – Список созданных сессий

Далее присоединимся к сессии используя команду: **screen -r 175133.top**

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
40445	eltex-pt	20	0	8296	2536	1840	S	0.0	0.1	0:00.01	screen
40446	eltex-pt	20	0	8028	4864	3536	S	0.0	0.1	0:00.00	bash
40473	eltex-pt	20	0	8032	4728	3408	S	0.0	0.1	0:00.00	bash
40633	eltex-pt	20	0	9096	3476	2000	S	0.0	0.1	1:14.20	screen
40634	eltex-pt	20	0	8036	4772	3444	S	0.0	0.1	0:00.25	bash
40796	eltex-pt	20	0	6724	3500	2528	S	0.0	0.1	0:09.21	watch
41060	eltex-pt	20	0	8448	2744	2012	S	0.0	0.1	0:00.02	screen
41061	eltex-pt	20	0	8036	4804	3476	S	0.0	0.1	0:00.23	bash
41345	eltex-pt	20	0	19016	10740	8920	S	0.0	0.3	0:00.06	systemd
41346	eltex-pt	20	0	169704	3964	0	S	0.0	0.1	0:00.00	(sd-pam)
41609	eltex-pt	20	0	8840	3164	1880	S	0.0	0.1	0:00.34	screen
41610	eltex-pt	20	0	8040	4676	3348	S	0.0	0.1	0:00.00	bash
41812	eltex-pt	20	0	8036	4708	3372	S	0.0	0.1	0:44.24	bash
42335	eltex-pt	20	0	8312	2596	1880	S	0.0	0.1	0:00.01	screen
42336	eltex-pt	20	0	8028	4780	3456	S	0.0	0.1	0:00.71	bash
42630	eltex-pt	20	0	8004	3736	3440	S	0.0	0.1	0:00.00	screen
42636	eltex-pt	20	0	8032	4836	3512	S	0.0	0.1	0:00.00	bash
42887	eltex-pt	20	0	8268	2564	1968	S	0.0	0.1	0:00.00	screen
42888	eltex-pt	20	0	8028	4820	3496	S	0.0	0.1	0:00.28	bash
42906	eltex-pt	20	0	6724	3416	2444	S	0.0	0.1	0:05.22	watch
42999	eltex-pt	20	0	8032	4796	3476	S	0.0	0.1	0:00.00	bash
43201	eltex-pt	20	0	8272	2496	1840	S	0.0	0.1	0:00.83	screen
43203	eltex-pt	20	0	11680	5308	3148	S	0.0	0.1	0:11.94	top
43206	eltex-pt	20	0	8028	4668	3492	S	0.0	0.1	0:00.00	bash
43250	eltex-pt	20	0	8028	4644	3468	S	0.0	0.1	0:00.00	bash
43284	eltex-pt	20	0	8028	4668	3492	S	0.0	0.1	0:00.00	bash
43952	eltex-pt	20	0	6724	3440	2468	S	0.0	0.1	0:08.70	watch
44276	eltex-pt	20	0	8136	2372	1840	S	0.0	0.1	0:00.00	screen
44277	eltex-pt	20	0	8028	4644	3472	S	0.0	0.1	0:00.00	bash
44327	eltex-pt	20	0	8028	4792	3464	S	0.0	0.1	0:00.00	bash
45057	eltex-pt	20	0	6724	3364	2416	S	0.0	0.1	0:08.76	watch

Рисунок 10 – Сессия top

Создадим новое окно в сессии **top** при помощи комбинации клавиш **CTRL + A**, затем **C**

```
eltex-pg2-v11@eltex-2026-spring:~$ |
```

Рисунок 11 – Созданное окно

Далее выполнил команду: **watch /usr/bin/vmstat**, в созданном окне, которая показывает сведения о состоянии системы

```
Every 2.0s: /usr/bin/vmstat

procs -----memory----- ---swap-- -----io----- -system-- -----cpu-----
 r  b   swpd   free   buff  cache   si   so    bi    bo    in   cs us sy id wa st
 2  0       0 1844712  80260 1117708    0    0     4    17   90  169  0  0 99  1  0
```

Рисунок 12 – Краткая сводка о состоянии системы

Получим список окон с помощью комбинации клавиш **CTRL + A + двойные кавычки**:

```
Num Name
 0 bash
 1 bash
```

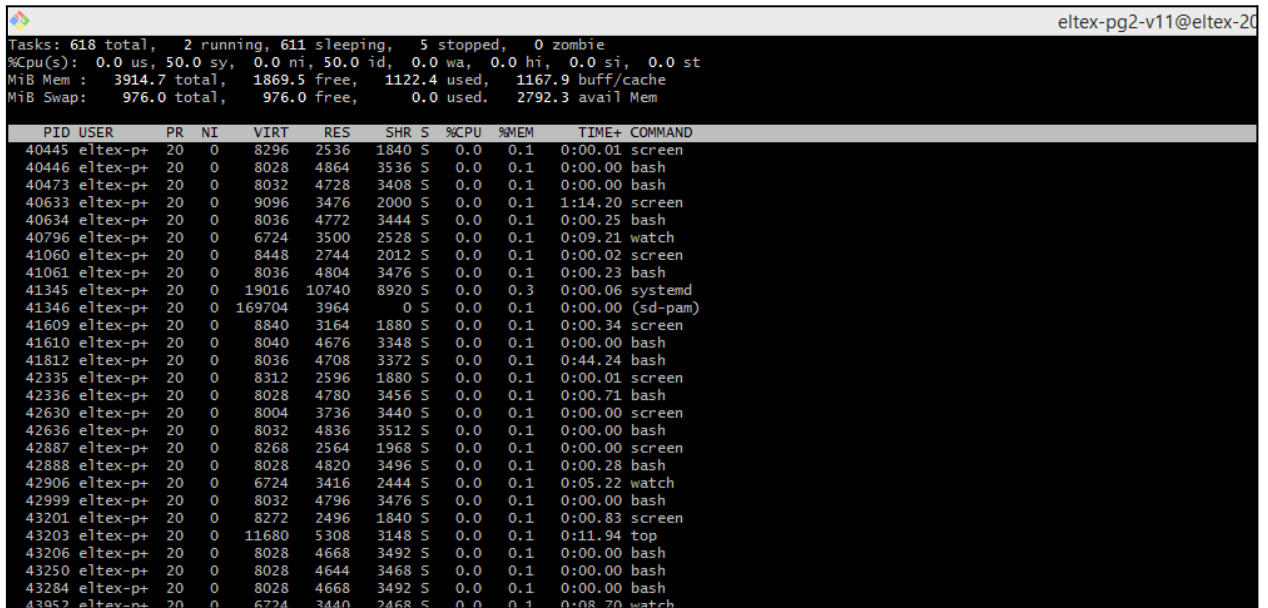
Рисунок 13 – Список окон

Далее переименуем окно *1 bash* в **vmstat**, используя комбинацию клавиш **CTRL + A**, затем **shift + A**

```
Num Name
 0 bash
 1 vmstat
```

Рисунок 14 – Переименованное окно в vmstat

Перейдем в окно с активным *top*:



```
Tasks: 618 total, 2 running, 611 sleeping, 5 stopped, 0 zombie
%Cpu(s): 0.0 us, 50.0 sy, 0.0 ni, 50.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 3914.7 total, 1869.5 free, 1122.4 used, 1167.9 buff/cache
MiB Swap: 976.0 total, 976.0 free, 0.0 used, 2792.3 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
40445	eltex-pt	20	0	8296	2536	1840	S	0.0	0.1	0:00.01	screen
40446	eltex-pt	20	0	8028	4864	3536	S	0.0	0.1	0:00.00	bash
40473	eltex-pt	20	0	8032	4728	3408	S	0.0	0.1	0:00.00	bash
40633	eltex-pt	20	0	9096	3476	2000	S	0.0	0.1	1:14.20	screen
40634	eltex-pt	20	0	8036	4772	3444	S	0.0	0.1	0:00.25	bash
40796	eltex-pt	20	0	6724	3500	2528	S	0.0	0.1	0:09.21	watch
41060	eltex-pt	20	0	8448	2744	2012	S	0.0	0.1	0:00.02	screen
41061	eltex-pt	20	0	8036	4804	3476	S	0.0	0.1	0:00.23	bash
41345	eltex-pt	20	0	19016	10740	8920	S	0.0	0.3	0:00.06	systemd
41346	eltex-pt	20	0	169704	3964	0	S	0.0	0.1	0:00.00	(sd-pam)
41609	eltex-pt	20	0	8840	3164	1880	S	0.0	0.1	0:00.34	screen
41610	eltex-pt	20	0	8040	4676	3348	S	0.0	0.1	0:00.00	bash
41812	eltex-pt	20	0	8036	4708	3372	S	0.0	0.1	0:44.24	bash
42335	eltex-pt	20	0	8312	2596	1880	S	0.0	0.1	0:00.01	screen
42336	eltex-pt	20	0	8028	4780	3456	S	0.0	0.1	0:00.71	bash
42630	eltex-pt	20	0	8004	3736	3440	S	0.0	0.1	0:00.00	screen
42636	eltex-pt	20	0	8032	4836	3512	S	0.0	0.1	0:00.00	bash
42887	eltex-pt	20	0	8268	2564	1968	S	0.0	0.1	0:00.00	screen
42888	eltex-pt	20	0	8028	4820	3496	S	0.0	0.1	0:00.28	bash
42906	eltex-pt	20	0	6724	3416	2444	S	0.0	0.1	0:05.22	watch
42999	eltex-pt	20	0	8032	4796	3476	S	0.0	0.1	0:00.00	bash
43201	eltex-pt	20	0	8272	2496	1840	S	0.0	0.1	0:00.83	screen
43203	eltex-pt	20	0	11680	5308	3148	S	0.0	0.1	0:11.94	top
43206	eltex-pt	20	0	8028	4668	3492	S	0.0	0.1	0:00.00	bash
43250	eltex-pt	20	0	8028	4644	3468	S	0.0	0.1	0:00.00	bash
43284	eltex-pt	20	0	8028	4668	3492	S	0.0	0.1	0:00.00	bash
43952	eltex-pt	20	0	6724	3440	2468	S	0.0	0.1	0:08.70	watch

Рисунок 15 – Окно с активным *top*

Далее отсоединимся от сессии, используя комбинацию клавиш **CTRL + A**, затем **D**:

```
[detached from 175133.top]
eltex-pg2-v11@eltex-2026-spring:~$ |
```

Рисунок 16 – Отсоединились от сессии

Далее создадим новую сессию:

```
[detached from 175133.top]
eltex-pg2-v11@eltex-2026-spring:~$ screen -r Solomon
There is a screen on:
      172723.Solomon (03/05/2026 07:36:18 PM) (Attached)
There is no screen to be resumed matching Solomon.
eltex-pg2-v11@eltex-2026-spring:~$ |
```

Рисунок 17 – Созданная новая сессия

В новом окне напишем команду: **vi report_part1.txt** для создания файла форматом txt

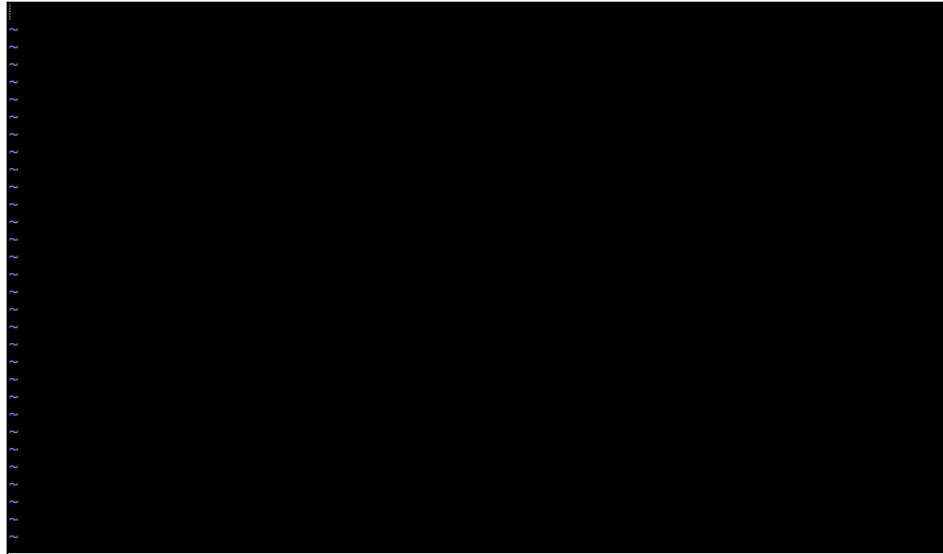


Рисунок 18 – Созданный txt файл

Далее проведем отключение от сессии

```
Type "screen [-d] -r [pid.]tty.host" to resume one of them.
eltex-pg2-v11@eltex-2026-spring:~$ screen -d 172723.Solomon
[172723.Solomon detached.]

eltex-pg2-v11@eltex-2026-spring:~$ |
```

Рисунок 19 – Отключение от сессии

Далее получим список сессий, используя команду: **screen -ls**

```
eltex-pg2-v11@eltex-2026-spring:~$ screen -ls
There are screens on:
  175133.top          (03/05/2026 07:37:59 PM)          (Detached)
  172723.Solomon      (03/05/2026 07:36:19 PM)          (Detached)
  167093.pts-98.eltex-2026-spring (03/05/2026 07:32:17 PM)          (Detached)
3 Sockets in /run/screen/S-eltex-pg2-v11.
eltex-pg2-v11@eltex-2026-spring:~$ |
```

Рисунок 20 – Список сессий

Далее отключаемся от сервера используя команду: **logout**

```
eltex-pg2-v11@eltex-2026-spring:~$ logout
Connection to 217.71.138.1 closed.

User@101-4 MINGW64 ~
$ |
```

Рисунок 21 – Отключение от сервера

Далее подключимся к серверу и получим список сессий:

```
There are screens on:
 175133.top      (03/05/2026 07:37:58 PM)      (Detached)
 172723.Solomon (03/05/2026 07:36:18 PM) (Detached)
 167093.pts-98.eltex-2026-spring (03/05/2026 07:32:16 PM) (Detached)
3 Sockets in /run/screen/S-eltex-pg2-v11.
eltex-pg2-v11@eltex-2026-spring:~$ |
```

Рисунок 22 – Список сессий

Далее зайдём в сессию с файлом и введём следующий текст:

```
Соломонов Дмитрий Михайлович, часть 1 модуль 1 успешно завершена 05.03.26 20:40
~
```

Рисунок 21 – Контент файла с расширением txt

Далее закроем все сессии, используя команду: **screen -S id session -X** и используем команду: **screen -r**, чтобы убедиться что нет активных сессий:

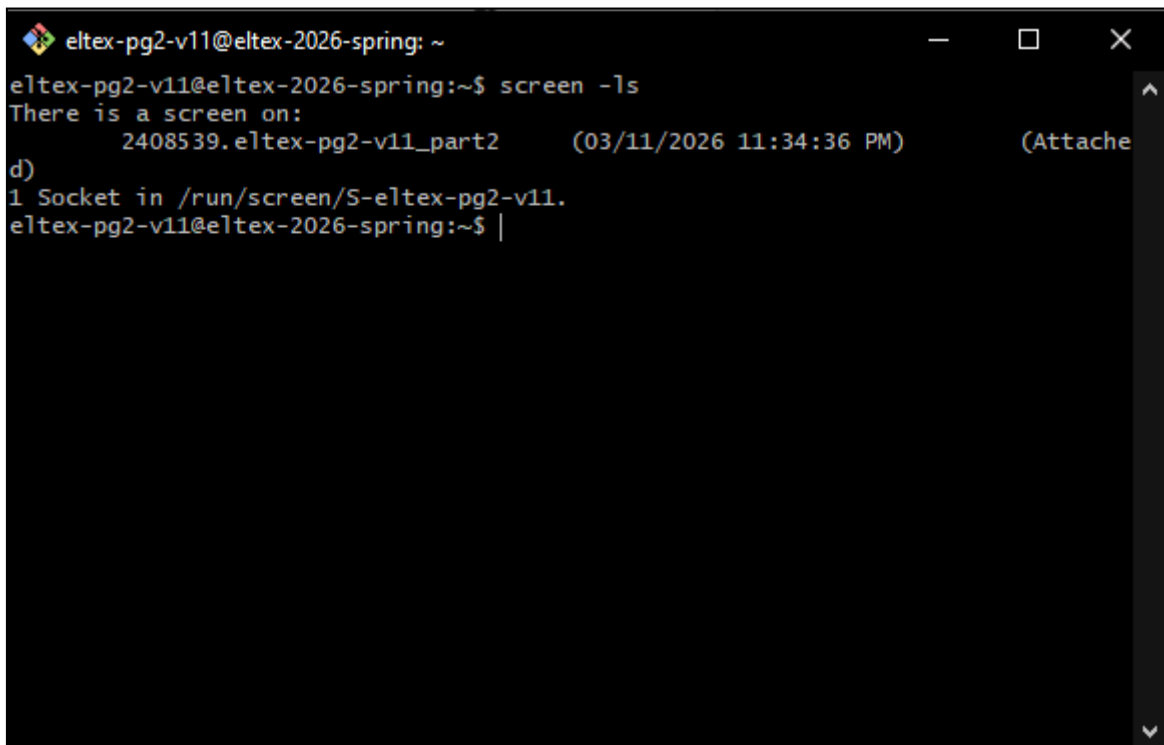
```
eltex-pg2-v11@eltex-2026-spring:~$ screen -r
There is no screen to be resumed.
eltex-pg2-v11@eltex-2026-spring:~$ |
```

Рисунок 22 – Отсутствие активных сессий

Выполнение лабораторной работы. Раздел № 2

Создадим новую сессию под именем "`whoami`_part2" и будем сохранять все команды в файл \$USER_part.log, который не содержит escape последовательностей. Для этого воспользуемся командой: **screen -L -Logfile "\${USER}_part.log" -S "\$(whoami)_part2"**. Далее используем команду **export TERM=dumb** для того чтобы “заставить” оболочку внутри screen работать в упрощенном режиме.

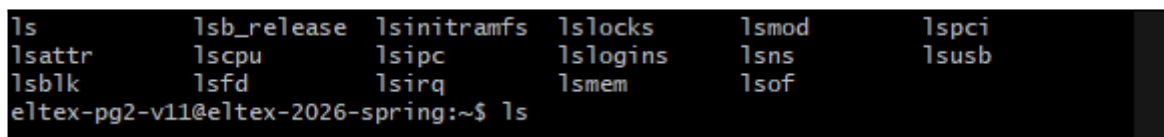
- -L – включает режим логирования в сессии.



```
eltex-pg2-v11@eltex-2026-spring: ~
eltex-pg2-v11@eltex-2026-spring:~$ screen -ls
There is a screen on:
      2408539.eltex-pg2-v11_part2      (03/11/2026 11:34:36 PM)      (Attache
d)
1 Socket in /run/screen/S-eltex-pg2-v11.
eltex-pg2-v11@eltex-2026-spring:~$ |
```

Рисунок 1 – Подключение к сессии

Далее выведем все команды, которые начинаются на ls. Для этого напечатаем команду в ls терминале, а затем нажмем клавишу Tab два раза.



```
ls      ls_release  lsinitramfs  lslocks    lsmod      lspci
lsattr   lscpu      lsipc        lslogins   lsns       lsusb
lsblk    lsfd       lsirq        lsmem      lsosf
eltex-pg2-v11@eltex-2026-spring:~$ ls
```

Рисунок 2 – Список всех команд, которые начинаются на ls

Далее выведем все переменные, которые начинаются с HIST. Для этого введем в командной строке \$HIST и затем нажмем клавишу Tab два раза.

```
$HISTCMD      $HISTCONTROL  $HISTFILE      $HISTFILESIZE  $HISTSIZE
eltex-pg2-v11@eltex-2026-spring:~$ $HIST
```

Рисунок 3 – Список всех переменных, которые начинаются с HIST

Для того чтобы узнать сколько команд может храниться в файле истории, выполним команду **echo \$HISTFILESIZE**.

```
$ echo $HISTFILESIZE
2000
```

Рисунок 4 – Количество команд, которое может храниться в файле истории

Далее выведем все имена файлов и каталогов из домашнего каталога, которые начинаются с «.». Для этого воспользуемся командой **ls -d ~/.***

```
/home/eltex-pg2-v11/.bash_history /home/eltex-pg2-v11/.config /home/eltex-pg2-v11/.viminfo
/home/eltex-pg2-v11/.bash_logout /home/eltex-pg2-v11/.log
/home/eltex-pg2-v11/.bashrc /home/eltex-pg2-v11/.profile
```

Рисунок 5 – Имена файлов и каталогов в домашнем каталоге, которые начинаются на .

Для настройки вывода даты выполнения команд используем следующую команду: **export HISTTIMEFORMAT="%d.%m.%y %H:%M:%S"** и для проверки результата используем команду **history**.

```
117 12.03.26 00:52:51ls -d /*
118 12.03.26 00:53:05ls -d ~/.*
119 12.03.26 00:54:27export HISTTIMEFORMAT="%d.%m.%y %H:%M:%S"
120 12.03.26 00:54:34history
121 12.03.26 00:57:59date
122 12.03.26 00:59:26history | tail -n 5
123 12.03.26 01:01:36tail -n 5 ~/.bash_history
124 12.03.26 01:04:18screen -ls
125 12.03.26 01:04:25screen -r
126 12.03.26 01:04:35screen -ls
127 12.03.26 01:05:14screen -d -r eltex-pg2-v11_part2
128 12.03.26 01:05:22y
129 12.03.26 01:05:24yes
130 12.03.26 01:08:14export TERM=dumb
131 12.03.26 01:08:14unalias ls
132 12.03.26 01:08:14PS1='$ '
133 12.03.26 01:09:02echo $HISTFILESIZE
134 12.03.26 01:09:14ls -d /*
135 12.03.26 01:09:47export HISTTIMEFORMAT="%d.%m.%y %H:%M:%S"
136 12.03.26 01:10:01history
```

Рисунок 6 – Вывод даты и времени для команд

Далее настроим автоматическое сохранение набираемых команд в файле истории. Для этого введем команду, например **date**. Далее проверим есть ли

данная команда в кеше и файле истории команд. Для этого используем команду **history | tail -n 5**.

```
133 12.03.26 01:09:02echo $HISTFILESIZE
134 12.03.26 01:09:14ls -d ~/.*
135 12.03.26 01:09:47export HISTTIMEFORMAT="%d.%m.%y %H:%M:%S"
136 12.03.26 01:10:01history
137 12.03.26 01:10:36history | tail -n 5
```

Рисунок 7 – Кэш истории команд

Посмотрим последние 5 строк в файле истории, используя команду: **tail -n 5 ~/.bash_history**.

```
screen -d -r eltex-pg2-v11_part2
#1773252322
y
#1773252324
yes
```

Рисунок 8 – Файл истории команд

В данном файле нет команд, так как они находятся еще в кеше и обычно bash пишет туда команды только после закрытия терминала(сессии).

Далее определим переменную **PROMPT_COMMAND**, чтобы кэш истории сохранялся в файле истории. Для этого используем команду: **export PROMPT_COMMAND='history -a'**.

Для проверки введем команду например echo "Hello World". Проверим кэш.

```
138 12.03.26 01:14:03export PROMPT_COMMAND='history -a'
139 12.03.26 01:15:57echo "Hello World"
140 12.03.26 01:16:25history | tail -n 3
```

Рисунок 9 – Кэш history

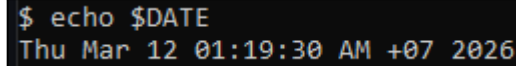
Проверим файлы на диске.

```
echo "Hello World"
#1773252985
history | tail -n 3
```

Рисунок 10 – Файл history

Команда появилась мгновенно, не дожидаясь закрытия терминала.

Далее создадим переменную `DATE` в которую запишем текущую дату. Для этого используем команду: `DATE=$(date)`. И далее используя команду: `echo $DATE`, выведем ее содержимое.

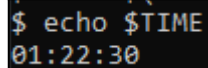


```
$ echo $DATE
Thu Mar 12 01:19:30 AM +07 2026
```

Рисунок 11 – Содержимое переменной `DATE`

Далее в переменную `TIME` запишем текущее время. Для этого воспользуемся командой: `TIME=$(date +%T)`. `%T` – чтобы вывести только часы, минуты, секунды.

Далее выведем содержимое переменной `TIME`.

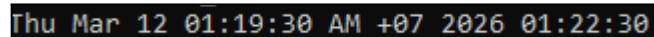


```
$ echo $TIME
01:22:30
```

Рисунок 12– Содержимое переменной `TIME`

Далее в переменную `DATE_TIME` поместим содержимое переменных `DATE`, `TIME` разделенные пробелом. Для этого воспользуемся командой: `DATE_TIME="$DATE $TIME"`

Проверим содержимое переменной `DATE_TIME`.



```
Thu Mar 12 01:19:30 AM +07 2026 01:22:30
```

Рисунок 13– Содержимое переменной `DATE_TIME`

Выведем имена файлов, содержащих хотя бы одну цифру из каталогов `/bin`, `/sbin`. Для этого используем команды: `ls /bin/*[0-9]*`, `ls /sbin/*[0-9]*`.

```
/bin/c89 /bin/pod2usage /bin/x86_64-linux-gnu-gcov
/bin/c89-gcc /bin/py3clean /bin/x86_64-linux-gnu-gcov-12
/bin/c99 /bin/py3compile /bin/x86_64-linux-gnu-gcov-dump
/bin/c99-gcc /bin/py3versions /bin/x86_64-linux-gnu-gcov-dump-12
/bin/cpan5.36-x86_64-linux-gnu /bin/pydoc3 /bin/x86_64-linux-gnu-gcov-tool
/bin/cpp-12 /bin/pydoc3.11 /bin/x86_64-linux-gnu-gcov-tool-12
/bin/diff3 /bin/pygettext3 /bin/x86_64-linux-gnu-gold
/bin/enc2xs /bin/pygettext3.11 /bin/x86_64-linux-gnu-gp-archive
/bin/gcc-12 /bin/python3 /bin/x86_64-linux-gnu-gp-collect-app
/bin/gcc-ar-12 /bin/python3.11 /bin/x86_64-linux-gnu-gp-display-html
/bin/gcc-nm-12 /bin/sha1sum /bin/x86_64-linux-gnu-gp-display-src
/bin/gcc-ranlib-12 /bin/sha224sum /bin/x86_64-linux-gnu-gp-display-text
/bin/gcov-12 /bin/sha256sum /bin/x86_64-linux-gnu-gprof
/bin/gcov-dump-12 /bin/sha384sum /bin/x86_64-linux-gnu-gprofng
/bin/gcov-tool-12 /bin/sha512sum /bin/x86_64-linux-gnu-ld
/bin/grub-menulst2cfg /bin/ssh-argv0 /bin/x86_64-linux-gnu-ld.bfd
/bin/grub-mkpasswd-pbkdf2 /bin/systemd-id128 /bin/x86_64-linux-gnu-ld.gold
/bin/grub-syslinux2cfg /bin/traceroute6 /bin/x86_64-linux-gnu-lto-dump
/bin/h2ph /bin/traceroute6.db /bin/x86_64-linux-gnu-lto-dump-12
/bin/h2xs /bin/x86_64 /bin/x86_64-linux-gnu-nm
/bin/i386 /bin/x86_64-linux-gnu-addr2line /bin/x86_64-linux-gnu-objcopy
/bin/linux32 /bin/x86_64-linux-gnu-ar /bin/x86_64-linux-gnu-objdump
/bin/linux64 /bin/x86_64-linux-gnu-as /bin/x86_64-linux-gnu-ranlib
/bin/lto-dump-12 /bin/x86_64-linux-gnu-c++filt /bin/x86_64-linux-gnu-readelf
/bin/md5sum /bin/x86_64-linux-gnu-cpp /bin/x86_64-linux-gnu-size
/bin/md5sum.textutils /bin/x86_64-linux-gnu-cpp-12 /bin/x86_64-linux-gnu-strings
/bin/pdb3 /bin/x86_64-linux-gnu-dwp /bin/x86_64-linux-gnu-strip
/bin/pdb3.11 /bin/x86_64-linux-gnu-elfedit
/bin/perl5.36.0 /bin/x86_64-linux-gnu-gcc
```

Рисунок 14 – Файлы содержащие хотя бы одну цифру в названии в каталоге `/bin`

```
$ ls /sbin/*[0-9]*
/sbin/dumpe2fs /sbin/e2label /sbin/e2undo /sbin/fsck.ext3 /sbin/mkfs.ext2 /sbin/tune2fs
/sbin/e2freefrag /sbin/e2mmpstatus /sbin/e4crypt /sbin/fsck.ext4 /sbin/mkfs.ext3 /sbin/update-grub2
/sbin/e2fsck /sbin/e2scrub /sbin/e4defrag /sbin/killall5 /sbin/mkfs.ext4
/sbin/e2image /sbin/e2scrub_all /sbin/fsck.ext2 /sbin/mke2fs /sbin/resize2fs
```

Рисунок 15 – Файлы содержащие хотя бы одну цифру в названии в каталоге `/sbin`

Далее изменим приглашение так, чтобы выводились имя хоста, имя пользователя и время: `имя_пользователя@имя_хоста-НН:ММ>`. Для этого воспользуемся командой: `PS1='\u@\h-$(date +"%H:%M")> '`

```
eltex-pg2-v11@eltex-2026-spring-01:30>
```

Рисунок 16 – Измененное приглашение

Далее сделаем так чтобы в запускаемом интерпретаторе `bash` выводилось приглашение, установленное в родительском интерпретаторе `bash`. Для этого экспортируем переменную `PS1`, используя команду **`export PS1`**. Для проверки запустим новый `bash`, используя команду **`bash -norc`**.

```
eltex-pg2-v11@eltex-2026-spring-01:37>
```

*Рисунок 16 – Приглашение установленное в родительском интерпретаторе
bash*

Создадим в домашнем каталоге подкаталоги для каждого месяца текущего года вида YYYY-MM. Для этого воспользуемся командой: `mkdir -p ~/ "$(date +%Y)"-{01..12}`

Проверим появились ли папки в домашнем каталоге. Для этого используем команду: `ls -d ~/2026-*`

```
/home/eltex-pg2-v11/2026-01 /home/eltex-pg2-v11/2026-04 /home/eltex-pg2-v11/2026-07 /home/eltex-pg2-v11/2026-10
/home/eltex-pg2-v11/2026-02 /home/eltex-pg2-v11/2026-05 /home/eltex-pg2-v11/2026-08 /home/eltex-pg2-v11/2026-11
/home/eltex-pg2-v11/2026-03 /home/eltex-pg2-v11/2026-06 /home/eltex-pg2-v11/2026-09 /home/eltex-pg2-v11/2026-12
```

Рисунок 17 – Созданные подкаталоги

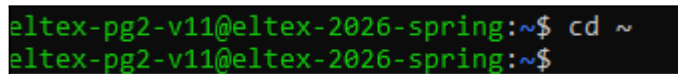
После закрытия сессии, посмотрим содержимое файла .log.

```
/bin/linux32 /bin/x86_64-linux-gnu-ar /bin/x86_64-linux-gnu-objdump
/bin/linux64 /bin/x86_64-linux-gnu-as /bin/x86_64-linux-gnu-ranlib
/bin/lto-dump-12 /bin/x86_64-linux-gnu-c++filt /bin/x86_64-linux-gnu-readelf
/bin/md5sum /bin/x86_64-linux-gnu-cpp /bin/x86_64-linux-gnu-size
/bin/md5sum.textutils /bin/x86_64-linux-gnu-cpp-12 /bin/x86_64-linux-gnu-strings
/bin/pdb3 /bin/x86_64-linux-gnu-dwp /bin/x86_64-linux-gnu-strip
/bin/pdb3.11 /bin/x86_64-linux-gnu-elfedit
/bin/perl5.36.0 /bin/x86_64-linux-gnu-gcc
$ ls /sbin/*[0-9]*
/sbin/dmcc2fs /sbin/e2label /sbin/e2undo /sbin/fsck.ext3 /sbin/mkfs.ext2 /sbin/tune2fs
/sbin/e2freefrag /sbin/e2mmpstatus /sbin/e4crypt /sbin/fsck.ext4 /sbin/mkfs.ext3 /sbin/update-grub2
/sbin/e2fsck /sbin/e2scrub /sbin/e4defrag /sbin/killall5 /sbin/mkfs.ext4
/sbin/e2image /sbin/e2scrub_all /sbin/fsck.ext2 /sbin/mke2fs /sbin/resize2fs
$ PS1='\u@\h-\t> '
eltex-pg2-v11@eltex-2026-spring-01:30:38> PS1='\u@\h-$(date +"%H:%M")> '
eltex-pg2-v11@eltex-2026-spring-01:30> export PS1
eltex-pg2-v11@eltex-2026-spring-01:34> bash
eltex-pg2-v11@eltex-2026-spring:~$ exit
exit
eltex-pg2-v11@eltex-2026-spring-01:36> bash --norc
eltex-pg2-v11@eltex-2026-spring-01:37> exit
exit
eltex-pg2-v11@eltex-2026-spring-01:38> mkdir -p ~/ "$(date +%Y)"-{01..12}
eltex-pg2-v11@eltex-2026-spring-01:40> ls -d ~/2026-*
/home/eltex-pg2-v11/2026-01 /home/eltex-pg2-v11/2026-04 /home/eltex-pg2-v11/2026-07 /home/eltex-pg2-v11/2026-10
/home/eltex-pg2-v11/2026-02 /home/eltex-pg2-v11/2026-05 /home/eltex-pg2-v11/2026-08 /home/eltex-pg2-v11/2026-11
/home/eltex-pg2-v11/2026-03 /home/eltex-pg2-v11/2026-06 /home/eltex-pg2-v11/2026-09 /home/eltex-pg2-v11/2026-12
eltex-pg2-v11@eltex-2026-spring-01:41> exit
```

Рисунок 18 – Содержимое файла .log

Выполнение лабораторной работы №1. Раздел № 3

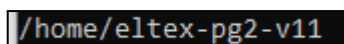
Для начала перейдем в домашней каталог, используя команду: ***cd ~***



```
eltex-pg2-v11@eltex-2026-spring:~$ cd ~
eltex-pg2-v11@eltex-2026-spring:~$
```

Рисунок 1 – Домашний каталог

Используя команду ***pwd***, посмотреть полный путь от корневого каталога, до рабочей директории где находится пользователь.

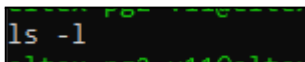


```
/home/eltex-pg2-v11
```

Рисунок 2 – Результат выполнения команды ***pwd***

Далее создадим файл ***\$USER_part3.log***. Для этого воспользуемся командой: ***touch "\${USER}_part3.log"***.

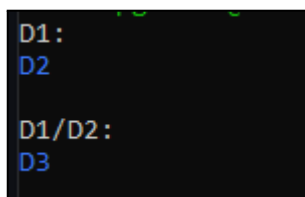
Для примера давайте запишем в лог команду ***ls -l***. Для этого воспользуемся командой: ***echo 'ls -l' >> "\${USER}_part3.log"*** и посмотрим содержимое файла, используя команду: ***cat \${USER}_part3.log***.



```
ls -l
```

Рисунок 3 – Содержимое файла ***_part3.log***

Далее создадим иерархию вложенных каталогов ***D1/D2/D3***. Для этого используем команду: ***mkdir -p D1/D2/D3***. Для проверки создания иерархии папок, воспользуемся командой: ***ls -R D1***.

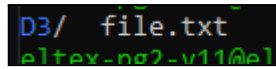


```
D1:
D2

D1/D2:
D3
```

Рисунок 3 – Содержимое файла ***_part3.log***

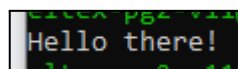
Далее в каталоге D2 создадим обычный файл с расширением .txt. Для этого воспользуемся командой: ***touch D1/D2/file.txt***. Для того чтобы проверить, что файл создан воспользуемся командой: ***ls -F D1/D2***.



```
D3/ file.txt
eltex-pg2-v11@el
```

Рисунок 4 – Содержимое каталога D2

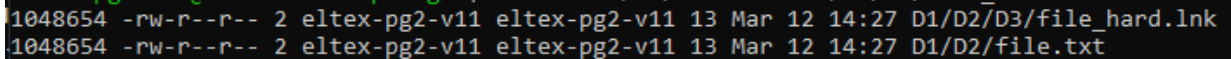
Добавим произвольный текст в файл file.txt. Для этого используем команду: ***cat D1/D2/file.txt***.



```
Hello there!
```

Рисунок 5 – Содержимое файла file.txt

Далее создадим жесткую ссылку на file.txt. Для этого выполним команду: ***ln D1/D2/file.txt D1/D2/D3/file_hard.lnk***. Для того чтобы проверить, что жесткая ссылка создана правильно. Воспользуемся командой: ***ls -li D1/D2/file.txt D1/D2/D3/file_hard.lnk***.

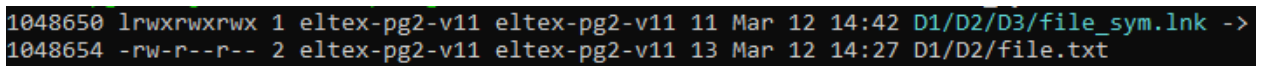


```
1048654 -rw-r--r-- 2 eltex-pg2-v11 eltex-pg2-v11 13 Mar 12 14:27 D1/D2/D3/file_hard.lnk
1048654 -rw-r--r-- 2 eltex-pg2-v11 eltex-pg2-v11 13 Mar 12 14:27 D1/D2/file.txt
```

Рисунок 6 – Номера inode

Как мы видим номера inode совпали, значит мы успешно создали hard link.

Далее создадим символическую ссылку. Для этого воспользуемся командой: ***ln -s ../file.txt D1/D2/D3/file_sym.lnk***. Для проверки успешного создания символической ссылки используем команду: ***ls -li D1/D2/file.txt D1/D2/D3/file_sym.lnk***.

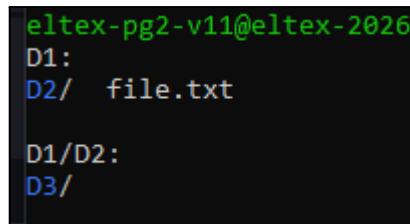


```
1048650 lrwxrwxrwx 1 eltex-pg2-v11 eltex-pg2-v11 11 Mar 12 14:42 D1/D2/D3/file_sym.lnk ->
1048654 -rw-r--r-- 2 eltex-pg2-v11 eltex-pg2-v11 13 Mar 12 14:27 D1/D2/file.txt
```

Рисунок 6 – Номера inode

Как мы видим, что символическая ссылка и оригинальный txt файл имеют разные номера inode, это говорит о том что мы успешно создали символическую ссылку на данный файл.

Далее переместим file.txt в каталог D1. Для этого воспользуемся командой: ***mv D1/D2/file.txt D1/*** и после проверим, содержание D1. Для этого воспользуемся командой: ***ls -RF D1***

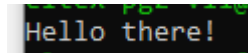


```
eltex-pg2-v11@eltex-2026
D1:
D2/  file.txt

D1/D2:
D3/
```

Рисунок 7 – Расположение файла file.txt в D1

Далее проверим работоспособность ссылок. Для начала проверим hard_link. Для этого воспользуемся командой: ***cat D1/D2/D3/file_hard.lnk***

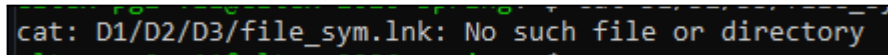


```
eltex-pg2-v11@eltex-2026
Hello there!
```

Рисунок 8 – Текст файла на который ссылка hard_link

Как мы видим жесткая ссылка указывает на данные записанные файл.

Далее проверим символическую ссылку. Для этого воспользуемся командой: ***cat D1/D2/D3/file_sym.lnk***



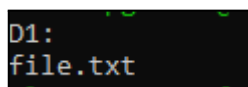
```
eltex-pg2-v11@eltex-2026
cat: D1/D2/D3/file_sym.lnk: No such file or directory
```

Рисунок 9 – file_sym.lnk не обнаружен

После перемещения файла file.txt символическая ссылка сломалась, так она хранит абсолютный путь до файла, который стал неправильным после его перемещения.

Далее удалим каталог D2 со всем его содержимым. Для этого воспользуемся командой: ***rm -rf D1/D2***

Проверяем успешно ли удалился каталог D2. Для этого воспользуемся командой: ***ls -RF D1***



```
eltex-pg2-v11@eltex-2026
D1:
file.txt
```

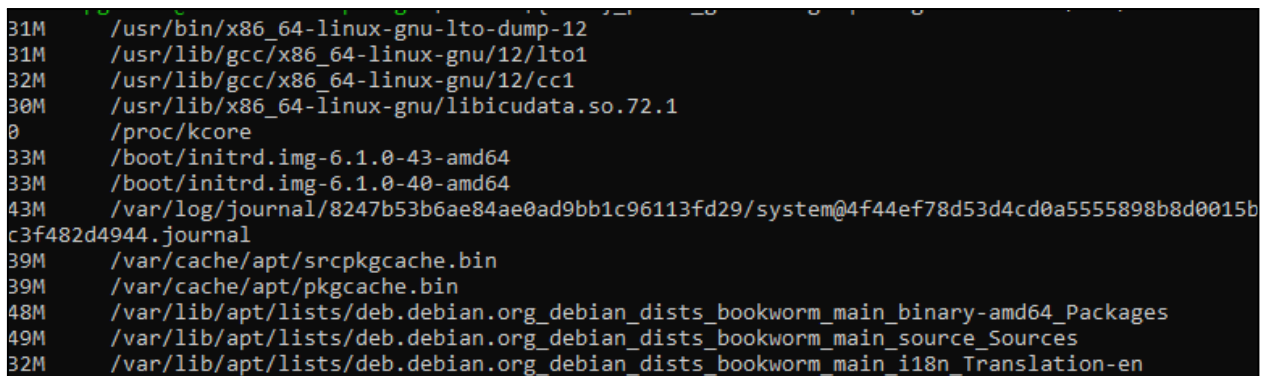
Рисунок 10 – Содержимое каталога D1

Как мы видим каталог D2 был удален.

Далее найдем все файлы в системе, которые имеют размер больше 20МБ и запишем в файл `$USER_part3_gt50M.log`, список найденных файлов. Для этого воспользуемся командой: `find / -type f -size +20M 2>/dev/null > "${USER}_part3_gt50M.log"`

- `2>/dev/null` – сокрытие сообщения о ошибках доступа. Так есть файлы, которые могут быть недоступны пользователю и для того чтобы лог был чистым, мы используем данную команду.

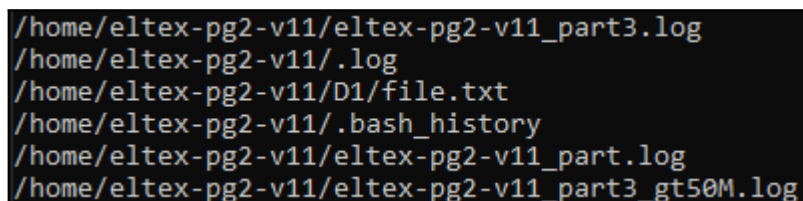
Для того чтобы убедиться, что файлы и вправду большие воспользуемся командой: `cat "${USER}_part3_gt50M.log" | xargs du -sh 2>/dev/null`



```
31M    /usr/bin/x86_64-linux-gnu-lto-dump-12
31M    /usr/lib/gcc/x86_64-linux-gnu/12/lto1
32M    /usr/lib/gcc/x86_64-linux-gnu/12/cc1
30M    /usr/lib/x86_64-linux-gnu/libcudata.so.72.1
0      /proc/kcore
33M    /boot/initrd.img-6.1.0-43-amd64
33M    /boot/initrd.img-6.1.0-40-amd64
43M    /var/log/journal/8247b53b6ae84ae0ad9bb1c96113fd29/system@4f44ef78d53d4cd0a5555898b8d0015b
c3f482d4944.journal
39M    /var/cache/apt/srcpkgcache.bin
39M    /var/cache/apt/pkgcache.bin
48M    /var/lib/apt/lists/deb.debian.org_debian_dists_bookworm_main_binary-amd64_Packages
49M    /var/lib/apt/lists/deb.debian.org_debian_dists_bookworm_main_source_Sources
32M    /var/lib/apt/lists/deb.debian.org_debian_dists_bookworm_main_i18n_Translation-en
```

Рисунок 11 – Размер каждого файла, который есть в `part3_gt50M.log`

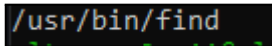
Далее в домашнем каталоге и его подкаталогах посмотрим обычные файлы, которые были изменены в течение последних 24 часов. Для этого воспользуемся командой: `find ~ -type f -mtime -1`



```
/home/eltex-pg2-v11/eltex-pg2-v11_part3.log
/home/eltex-pg2-v11/.log
/home/eltex-pg2-v11/D1/file.txt
/home/eltex-pg2-v11/.bash_history
/home/eltex-pg2-v11/eltex-pg2-v11_part.log
/home/eltex-pg2-v11/eltex-pg2-v11_part3_gt50M.log
```

Рисунок 12 – Файлы, которые были изменены в течение последних 24 часов

Чтобы посмотреть в каком каталоге находится команда `find` воспользуемся командой: ***which find***



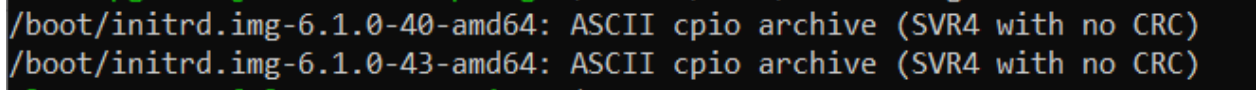
```
/usr/bin/find
```

Рисунок 13 – Расположение команды `find`

Определим характер содержимого `find` командой: ***file /usr/bin/find***

Получим следующие: `/usr/bin/find: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=5545a6c7162d325fbe5ffabcbfe4e5521b66ce70, for GNU/Linux 3.2.0, stri`

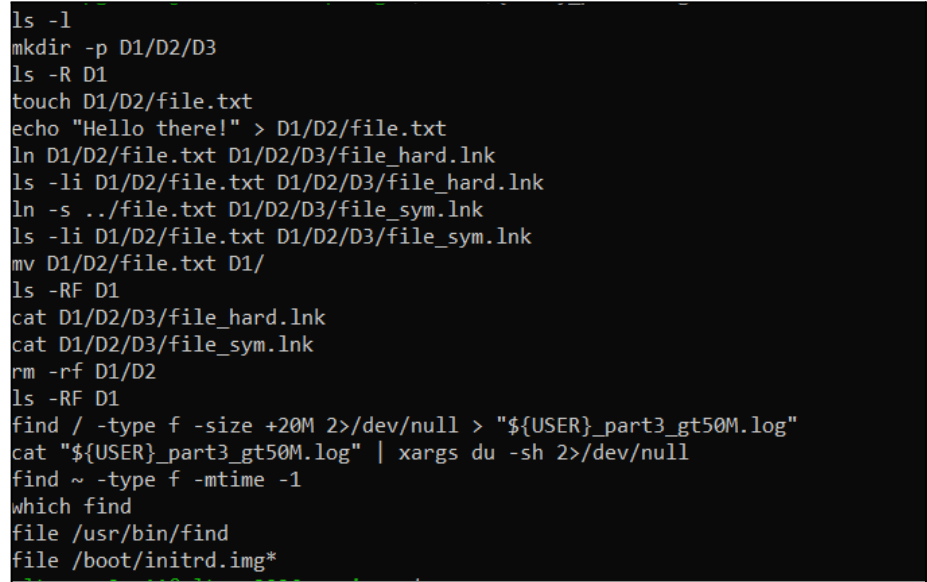
Далее установим к какому типу относятся файлы `/boot/initrd.img*`. Для этого воспользуемся командой: ***file /boot/initrd.img****



```
/boot/initrd.img-6.1.0-40-amd64: ASCII cpio archive (SVR4 with no CRC)
/boot/initrd.img-6.1.0-43-amd64: ASCII cpio archive (SVR4 with no CRC)
```

Рисунок 14 – Тип файлов `/boot/initrd.img*`

Далее проверим файл с логами. Для этого воспользуемся командой: ***cat \${USER}_part3.log***



```
ls -l
mkdir -p D1/D2/D3
ls -R D1
touch D1/D2/file.txt
echo "Hello there!" > D1/D2/file.txt
ln D1/D2/file.txt D1/D2/D3/file_hard.lnk
ls -li D1/D2/file.txt D1/D2/D3/file_hard.lnk
ln -s ../file.txt D1/D2/D3/file_sym.lnk
ls -li D1/D2/file.txt D1/D2/D3/file_sym.lnk
mv D1/D2/file.txt D1/
ls -RF D1
cat D1/D2/D3/file_hard.lnk
cat D1/D2/D3/file_sym.lnk
rm -rf D1/D2
ls -RF D1
find / -type f -size +20M 2>/dev/null > "${USER}_part3_gt50M.log"
cat "${USER}_part3_gt50M.log" | xargs du -sh 2>/dev/null
find ~ -type f -mtime -1
which find
file /usr/bin/find
file /boot/initrd.img*
```

Рисунок 15 – Содержимое файла `${USER}_part3.log`

Лабораторная работа №2. Раздел №1

Создадим на сервере 1 ключ ssh при помощи программы ssh-keygen.

```
eltex-pg2-v11@eltex-2026-spring:~$ ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/home/eltex-pg2-v11/.ssh/id_rsa):
Created directory '/home/eltex-pg2-v11/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/eltex-pg2-v11/.ssh/id_rsa
Your public key has been saved in /home/eltex-pg2-v11/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:SEduhz7faXQpsxG2mpcGuV0SaIRVL351zivrz4tBZZ0 eltex-pg2-v11@eltex-2026-spring
The key's randomart image is:
+---[RSA 3072]---+
|      .oo.      |
|     o.o . . o  |
|    . = + = .E+ |
|   . = o + =o=. |
|  . S o O.= o   |
|   o O.% .      |
|    = Xo .      |
|     + *        |
|      .+.+.     |
+---[SHA256]-----+
eltex-pg2-v11@eltex-2026-spring:~$ S
```

Рисунок 1 – Создание ssh ключа

Копируем созданный ключ на сервер 2, используя команду: ssh-key-id
root@172.16.9.211

Теперь подключимся ко второму серверу для сравнения содержимого ключей
на сервере 1 и 2.

```
Last login: Thu Mar 12 10:46:09 2026 from 172.16.8.4
root@eltex-practice2-pg2-v11:~# cat ~/.ssh/authorized_keys
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQGDHFRfo81U5L8bvhm10ejQKfUKYIwkuB2TYME1BpiUFnpLWE/MsUajm8buJ9pP0EFORfI7ExeLbm14s/PI
My1F//LbGKqaSwWJ6+9c2K91NNkL4yHAv3G5D+Akjek0ciB1D12j5WrYGAExwIj40cTFZMS/dwGz8/XHtp2ABhndzYrGg7MEPMQ4ov0ti4Gazx5iwsPb
bHa01Fne0= eltex-pg2-v11@eltex-2026-spring
root@eltex-practice2-pg2-v11:~# |
```

Рисунок 2 – Содержимое ключа ssh для сервера 2

```
eltex-pg2-v11@eltex-2026-spring:~$ cat ~/.ssh/*.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQGDHFRfo81U5L8bvhm10ejQKfUKYIwkuB2TYME1BpiUFnpLWE/MsUajm8buJ9pP0EFORfI7ExeLbm14s/PI
RSXKUM2HYrcTarXec1VOEwfdG5zC1CgBtNB4dm0pFhiGyG6K6czaYTFzaHj+04J0mBDweeyjZ0mhHGuk/My1F//LbGKqaSwWJ6+9c2K91NNkL4yHAv3G5D
azx5iwsPbSOYExxNpOosNOU+4StxpB8AoObCwI1vbthYRp68NtE9PI+hjuqLjJ181pJyRpw+ExVAUOKWGuk8ZQ/xrFUGWS63As2ax1uATWIA4sKjH1SF8H
ex-2026-spring
eltex-pg2-v11@eltex-2026-spring:~$ |
```

Рисунок 3 – Содержимое ключа ssh для сервера 1

Как мы видим содержимое ключа ssh для серверов 1, 2 идентичны, это говорит о том что мы скопировали содержимое ключа ssh с сервера 1 на сервер 2 успешно.

Также проверим права доступа к файлам на 1 и 2 сервере и сравним их.

```
eltex-pg2-v11@eltex-2026-spring:~$ ls -l ~/.ssh/*.pub
-rw-r--r-- 1 eltex-pg2-v11 eltex-pg2-v11 585 Mar 12 17:20 /home/eltex-pg2-v11/.ssh/id_rsa.pub
eltex-pg2-v11@eltex-2026-spring:~$ |
```

Рисунок 4 – Права доступа к файлу с ssh ключом на 1 сервере

```
root@eltex-practice2-pg2-v11:~# ls -l ~/.ssh/authorized_keys
-rw----- 1 root root 585 Mar 12 10:48 /root/.ssh/authorized_keys
root@eltex-practice2-pg2-v11:~# |
```

Рисунок 5 – Права доступа к файлу с ssh ключом на 2 сервере

На сервере 2 создадим нового пользователя user1 используя команду useradd и добавим пароль пользователю используя команду passwd user1

```
Last login: Thu Mar 12 10:50:12 2026 from 172.16.8.4
root@eltex-practice2-pg2-v11:~# ls -l ~/.ssh/authorized_keys
-rw----- 1 root root 585 Mar 12 10:48 /root/.ssh/authorized_keys
root@eltex-practice2-pg2-v11:~# useradd -m -s /bin/bash user1
root@eltex-practice2-pg2-v11:~# passwd user1
New password:
Retype new password:
passwd: password updated successfully
root@eltex-practice2-pg2-v11:~# |
```

Рисунок 6 – Создание user1 и присвоение ему пароля

Создадим user2, user3 используя команду useradd

```
info: Adding user `user2' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group `user2' (1002) ...
info: Adding new user `user2' (1002) with group `user2 (1002)' ...
info: Creating home directory `/home/user2' ...
info: Copying files from `/etc/skel' ...
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for user2
Enter the new value, or press ENTER for the default
    Full Name []:
    Room Number []:
    Work Phone []:
    Home Phone []:
```

Рисунок 7 – Создание user2 и присвоение ему пароля

```
root@eltex-practice2-pg2-v11:~# adduser user3
info: Adding user `user3' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group `user3' (1003) ...
info: Adding new user `user3' (1003) with group `user3 (1003)' ...
info: Creating home directory `/home/user3' ...
info: Copying files from `/etc/skel' ...
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for user3
Enter the new value, or press ENTER for the default
    Full Name []:
    Room Number []:
    Work Phone []:
    Home Phone []:
    Other []:
```

Рисунок 8 – Создание user3 и присвоение ему пароля

Для пользователя user3 смените shell на /usr/sbin/nologin (man usermod)

```
root@eltex-practice2-pg2-v11:~#
root@eltex-practice2-pg2-v11:~# usermod -s /usr/sbin/nologin user3
root@eltex-practice2-pg2-v11:~#
```

Рисунок 9 – смена оболочки для user3

Выполним вход под данным пользователь используя команду: **su -s /bin/bash user3**

```
this account is currently not available.
root@eltex-practice2-pg2-v11:~# su -s /bin/bash user3
user3@eltex-practice2-pg2-v11:/root$
```

Рисунок 10 – вход под user3

Далее выполним выход из user3, используя команду `exit`

```
user3@eltex-practice2-pg2-v11:/root$ exit
exit
```

Рисунок 11 - exit

Создадим новую группу добавьте её для всех пользователей user* как дополнительную, посмотрите список групп всех пользователей user*, используя команду: ***groupadd Boychiki***

Дальше добавим всех юзеров в данную группу, используя команду: ***usermod -aG Boychiki user****

```
root@eltex-practice2-pg2-v11:~# usermod -aG Boychiki user1
root@eltex-practice2-pg2-v11:~# usermod -aG Boychiki user2
root@eltex-practice2-pg2-v11:~# usermod -aG Boychiki user3
root@eltex-practice2-pg2-v11:~# |
```

Рисунок 12 - Добавление users в группу

Далее посмотрим в каких группах состоят пользователи. Для этого используем команду: ***id user****

```
uid=1001(user1) gid=1001(user1) groups=1001(user1),1004(Boychiki)
root@eltex-practice2-pg2-v11:~# id user2
uid=1002(user2) gid=1002(user2) groups=1002(user2),100(users),1004(Boychiki)
root@eltex-practice2-pg2-v11:~# id user3
uid=1003(user3) gid=1003(user3) groups=1003(user3),100(users),1004(Boychiki)
```

Рисунок 13 - Проверка групп в которых состоят пользователи

Создадим каталог /opt/share и назначим группу из предыдущего пункта его владельцем, установите на этот каталог бит SGID, права для группы rwx. Для этого используем команды: **mkdir -p /opt/share**, **chgrp Boychiki /opt/share**, **chmod 2770 /opt/share**

```
root@eltex-practice2-pg2-v11:~# id user2
uid=1002(user2) gid=1002(user2) groups=1002(user2),100(users),1004(Boychiki)
root@eltex-practice2-pg2-v11:~# id user3
uid=1003(user3) gid=1003(user3) groups=1003(user3),100(users),1004(Boychiki)
root@eltex-practice2-pg2-v11:~# mkdir -p /opt/share
root@eltex-practice2-pg2-v11:~# ls -l
total 0
root@eltex-practice2-pg2-v11:~# ls
root@eltex-practice2-pg2-v11:~# chgrp Boychiki /opt/share
root@eltex-practice2-pg2-v11:~# chmod 2770 /opt/share
root@eltex-practice2-pg2-v11:~# |
```

Рисунок 14 - Создание каталога и назначение владельца и установка бита SGID, права для группы rwx

Для user1 задайте permanently umask, снимающий право чтения для «прочих». Для этого используем команду: **vi /home/user1/.bashrc**

```
# See /usr/share/doc/bash-doc/examples in the bash-doc package.

if [ -f ~/.bash_aliases ]; then
    . ~/.bash_aliases
fi

# enable programmable completion features (you don't need to enable
# this, if it's already enabled in /etc/bash.bashrc and /etc/profile
# sources /etc/bash.bashrc).
if ! shopt -oq posix; then
    if [ -f /usr/share/bash-completion/bash_completion ]; then
        . /usr/share/bash-completion/bash_completion
    elif [ -f /etc/bash_completion ]; then
        . /etc/bash_completion
    fi
fi
umask 0027
"/home/user1/.bashrc" 118L, 3782B
```

Рисунок 15 - Добавим маску permanently

Создадим каждым из пользователей новые файлы в каталоге /opt/share, удалите файлы созданные другими пользователями, используя команды: ***su -user*, touch /opt/share/file1_by_user****

Например зайдём под вторым пользователем и удалим файл, который создали под первым пользователем. Для этого воспользуемся командой: ***rm /opt/share/file1_by_user1***

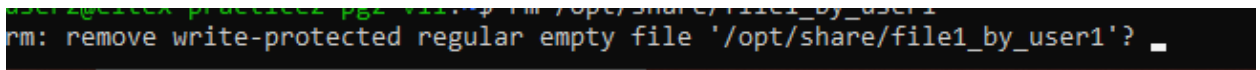


Рисунок 16 - Подтверждение на удаление файла

Система спрашивает подтверждение, потому что файл защищен от записи для текущего пользователя (user2), но так как у нас есть права на запись в саму папку /opt/share, то мы все равно имеем право его удалить.

Далее установим Sticky Bit на каталог /opt/share. Для этого используем команду: ***chmod 3770 /opt/share***

Теперь перейдем в user1 и создадим файл и дальше перейдем в user2 и попытаемся удалить файл, который был создан под user1 и в результате получим следующие:

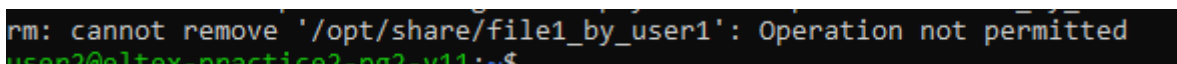


Рисунок 17 - Отказ в удаление файла

Это происходит по той причине, что Sticky Bit, разрешает удаление только владельцу файла или root.

Далее разрешим user1 выполнять привилегированную команду dmesg при помощи команды sudo, а user2 – при помощи скрипта на языке bash с установленным флагом SUID. Для этого необходимо отредактировать файл /etc/sudoers. Для этого воспользуемся командой ***visudo*** и добавим в конец файла следующую строку: ***user1 ALL=(ALL:ALL) /usr/bin/dmesg***

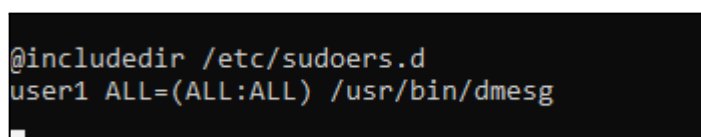


Рисунок 18 - Запись в файл /etc/sudoers

Далее выполним команду: ***sudo dmesg***

```
[87019.748496] systemd[1]: Started systemd-networkd.service - Network Configuration.
[87019.748798] systemd[1]: systemd-udevd.service: Deactivated successfully.
[87019.750443] systemd[1]: Stopped systemd-udevd.service - Rule-based Manager for Device Events and Files.
[87019.754205] systemd[1]: udisks2.service: Deactivated successfully.
[87019.755759] systemd[1]: Stopped udisks2.service - Disk Manager.
[87019.769216] systemd[1]: Starting systemd-networkd-wait-online.service - Wait for Network to be Configured...
[87019.786446] systemd[1]: Starting systemd-udevd.service - Rule-based Manager for Device Events and Files...
[87019.796172] systemd[1]: Starting udisks2.service - Disk Manager...
[87019.796312] systemd[1]: Stopping upower.service - Daemon for power management...
[87019.796633] systemd[1]: Started systemd-timesyncd.service - Network Time Synchronization.
[87019.802275] systemd[1]: Started ModemManager.service - Modem Manager.
[87019.812174] systemd[1]: Starting systemd-hostnamed.service - Hostname Service...
[87019.812367] systemd[1]: upower.service: Deactivated successfully.
[87019.812529] systemd[1]: Stopped upower.service - Daemon for power management.
[87019.814490] systemd[1]: Finished systemd-networkd-wait-online.service - Wait for Network to be Configured.
[87019.823203] systemd[1]: Starting upower.service - Daemon for power management...
[87019.823325] systemd[1]: Started systemd-resolved.service - Network Name Resolution.
[87019.851071] systemd[1]: Started systemd-udevd.service - Rule-based Manager for Device Events and Files.
```

Рисунок 19 – Кольцевой буфер сообщений ядра

Далее создадим bash-скрипт, чтобы пользователь user2 мог запускать dmesg с установленным SUID-битом. Для этого воспользуемся командой: ***nano /usr/local/bin/run_dmesg.sh***. После впишем следующие в файл: ***#!/bin/bash***
/usr/bin/dmesg

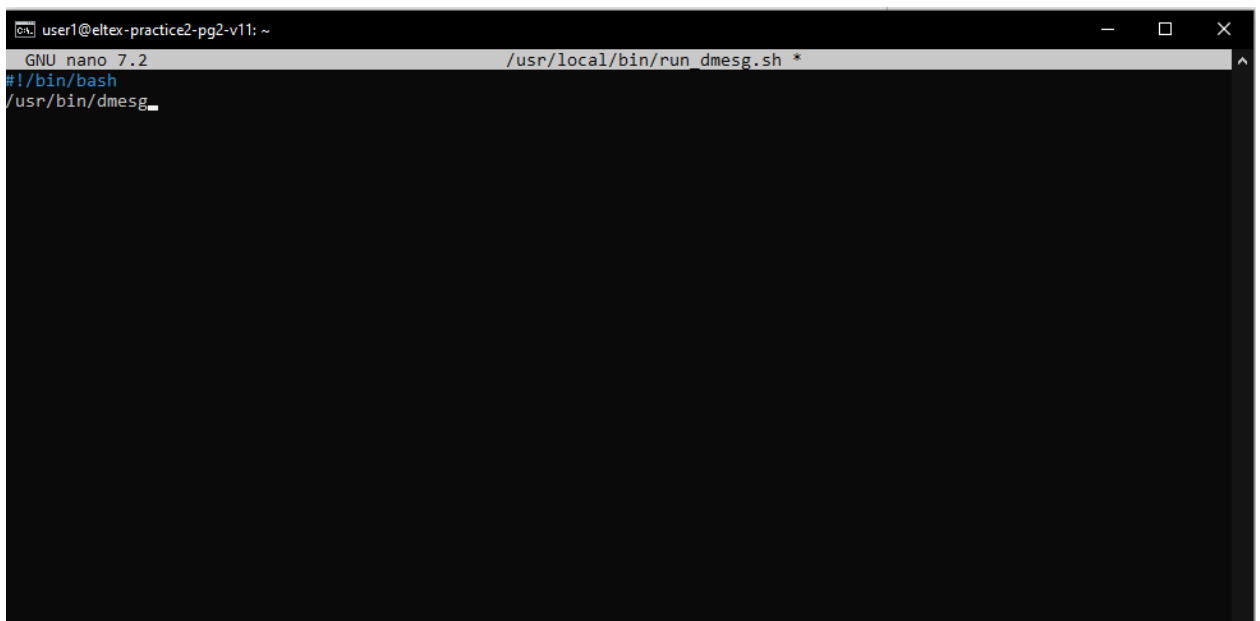
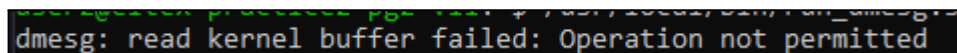


Рисунок 20 – Файл для записи

Далее установим владельца root. Используя команду: **chown root /usr/local/bin/run_dmesg.sh**

SUID-бит установим командой: **chmod 4755 /usr/local/bin/run_dmesg.sh**

Теперь проверим работу. Для этого зайдём под пользователем user2 и запустим скрипт, используя команду: **/usr/local/bin/run_dmesg.sh**



```
dmesg: read kernel buffer failed: Operation not permitted
```

Рисунок 21 – Доступ запрещен

В современных версиях Linux (ядрах выше 4.0) SUID-бит игнорируется для bash-скриптов из соображений безопасности. Ядро видит, что запускается интерпретатор (/bin/bash), и сбрасывает привилегии root до обычного пользователя.

Для всех пользователей user* зададим действие пароля 10 дней. Для этого зададим воспользуемся командой: **chage -M 10 user1, chage -M 10 user2, chage -M 10 user3,**

Отредактируем файл /etc/motd, вписав туда Соломонов Дмитрий. Для этого воспользуемся командой: **nano /etc/motd**

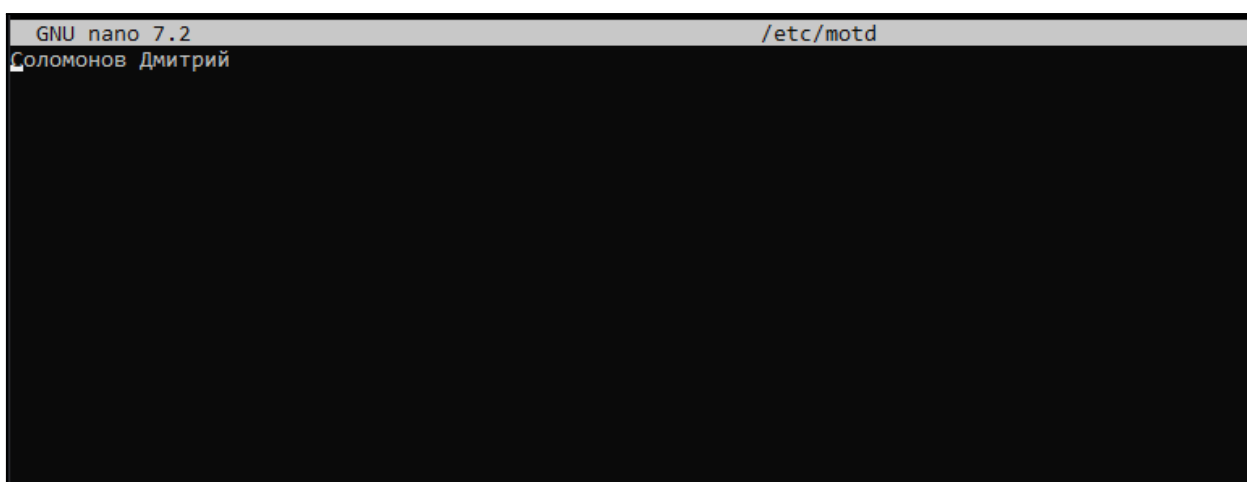
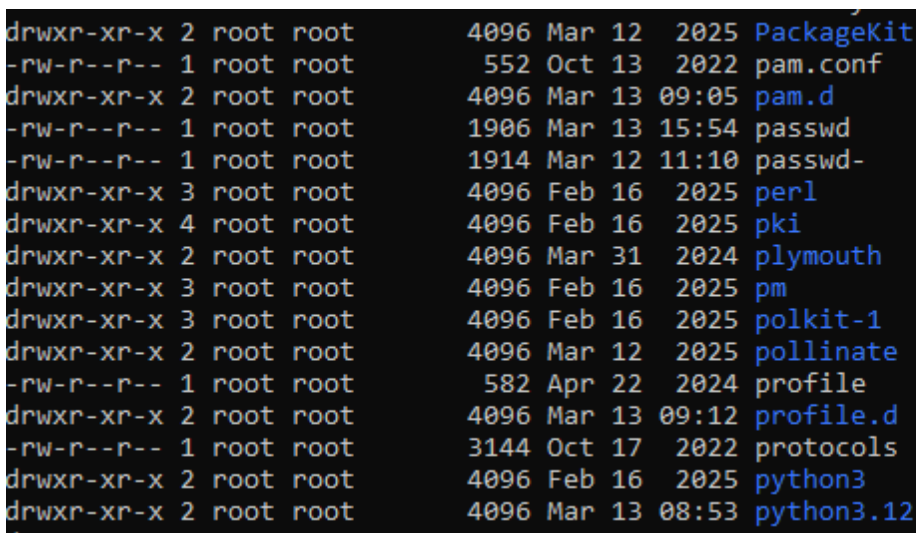


Рисунок 22 – Редактирование файла

Создадим копию содержимого каталога /etc в каталог /root/etc_backup при помощи программы rsync. Для начала создадим каталог etc_backup, используя команду: `mkdir -p /root/etc_backup`. Далее запустим копирование, используя команду: `rsync -av /etc/ /root/etc_backup/`

Проверяем успешность операции копирования:



```
drwxr-xr-x 2 root root      4096 Mar 12  2025 PackageKit
-rw-r--r-- 1 root root       552 Oct 13  2022 pam.conf
drwxr-xr-x 2 root root      4096 Mar 13 09:05 pam.d
-rw-r--r-- 1 root root      1906 Mar 13 15:54 passwd
-rw-r--r-- 1 root root      1914 Mar 12 11:10 passwd-
drwxr-xr-x 3 root root      4096 Feb 16  2025 perl
drwxr-xr-x 4 root root      4096 Feb 16  2025 pki
drwxr-xr-x 2 root root      4096 Mar 31  2024 plymouth
drwxr-xr-x 3 root root      4096 Feb 16  2025 pm
drwxr-xr-x 3 root root      4096 Feb 16  2025 polkit-1
drwxr-xr-x 2 root root      4096 Mar 12  2025 pollinate
-rw-r--r-- 1 root root       582 Apr 22  2024 profile
drwxr-xr-x 2 root root      4096 Mar 13 09:12 profile.d
-rw-r--r-- 1 root root     3144 Oct 17  2022 protocols
drwxr-xr-x 2 root root      4096 Feb 16  2025 python3
drwxr-xr-x 2 root root      4096 Mar 13 08:53 python3.12
```

Рисунок 23 – Скопированные файлы

Разархивируем содержимое каталога /root/etc_backup архиватором tar, используйте алгоритмы сжатия gzip, bzip2, 7zip, сравните размеры полученных файлов. Для этого используем команды:

tar -czf /root/etc_backup.tar.gz -C /root/etc_backup .

tar -cjf /root/etc_backup.tar.bz2 -C /root/etc_backup .

7z a /root/etc_backup.7z /root/etc_backup/

Далее сравним их размеры, используя команду: `ls -lh /root/etc_backup.tar.gz /root/etc_backup.tar.bz2 /root/etc_backup.7z`

```

Scan WARNINGS: 3
root@eltex-practice2-pg2-v11:~# ls -lh /root/etc_backup.tar.gz /root/etc_backup.tar.bz2 /root/etc_backup.7z
-rw-r--r-- 1 root root 4.5M Mar 13 16:48 /root/etc_backup.7z
-rw-r--r-- 1 root root 548K Mar 13 16:48 /root/etc_backup.tar.bz2
-rw-r--r-- 1 root root 583K Mar 13 16:45 /root/etc_backup.tar.gz
root@eltex-practice2-pg2-v11:~#

```

Рисунок 24 – Размеры архивов

Отредактируем файл `/etc/motd`, вписав туда текущую дату и время, синхронизируйте каталог `/root/etc_backup` с каталогом `/etc` при помощи `rsync`, добавьте файл `motd` в архив, сжатый `gzip`.

Добавим дату и время используя следующую команду: **`date >> /etc/motd`**

Теперь синхронизируем каталоги, чтобы туда попал обновленный файл `motd`. Для этого используем команду: **`rsync -av --delete /etc/ /root/etc_backup/`**

Добавим файл `motd` в архив сжатый `gzip`. Для этого используем команду: **`tar -czf /root/etc_backup.tar.gz -C /root/etc_backup .`**

Далее проверим содержимое архива, используя команду: **`tar -tvf /root/etc_backup.tar.gz | grep motd`**

```

ysinfo.wrapper
-rwxr-xr-x root/root      296 2025-02-16 21:04 ./update-motd.d/91-contract-ua-esm-status
-rwxr-xr-x root/root       84 2025-02-16 21:04 ./update-motd.d/85-fwupd
-rwxr-xr-x root/root      218 2025-02-16 21:04 ./update-motd.d/90-updates-available
-rwxr-xr-x root/root      558 2024-06-21 20:19 ./update-motd.d/91-release-upgrade
-rw-r--r-- root/root       66 2026-03-13 16:54 ./motd
-rw-r--r-- root/root      682 2025-02-16 21:04 ./default/motd-news
root@eltex-practice2-pg2-v11:~#

```

Рисунок 25 – Содержимое архива и `motd`

Сравним содержимое архива, упакованного `bzip2` с содержимым каталога `/root/etc_backup`. Для этого используем команду: **`tar --diff -jf /root/etc_backup.tar.bz2 -C /root/etc_backup .`**

```

root@eltex-practice2-pg2-v11:~/etc_backup# find /root/etc_backup -name '*' -type f
root@eltex-practice2-pg2-v11:~/etc_backup# tar --diff -jf /root/etc_backup.tar.bz2 -C /root/etc_backup .
root@eltex-practice2-pg2-v11:~/etc_backup#

```

Рисунок 26 – Результат команды `tar --diff -jf...`

Если команда ничего не вывела, то значит содержимое архива и каталога идентично.

Распакуем архивы `etc_backup`, упакованные `gzip` и `7zip` в каталоги `/root/etc_backup_gzip` и `/root/etc_backup_7zip`, сравните программой `diff` файлы

motd в этих каталогах. Для начала создадим директорию для распаковки. Для этого воспользуемся командами:

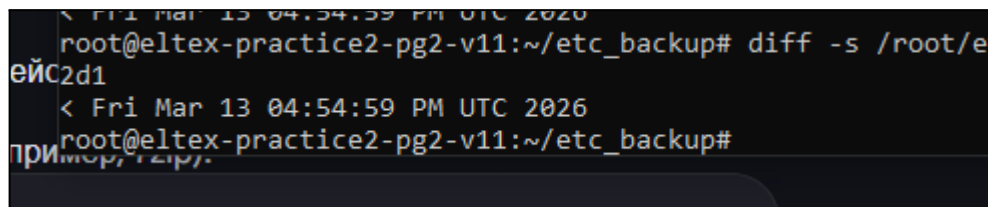
```
mkdir /root/etc_backup_gzip
```

```
mkdir /root/etc_backup_7zip
```

Далее распакуем Gzip архив, используя команду: **tar -xzf /root/etc_backup.tar.gz -C /root/etc_backup_gzip**

Также распакуем 7-zip архив, используя команду: **7z x /root/etc_backup.7z -o/root/etc_backup_7zip**

Далее сравним файлы motd, используя команду: **diff /root/etc_backup_gzip/motd /root/etc_backup_7zip/motd**



```
< Fri Mar 13 04:54:59 PM UTC 2026
root@eltex-practice2-pg2-v11:~/etc_backup# diff -s /root/e
ейс2d1
< Fri Mar 13 04:54:59 PM UTC 2026
root@eltex-practice2-pg2-v11:~/etc_backup#
```

Рисунок 27 – Разное содержимое файлов

Программа diff показывает что файлы различаются. Строка, которая начинается на < указывает на то что есть в архиве gzip и нет в архиве 7z

Выполнение лабораторной работы №2. Раздел № 2

Найдем все записи из лога загрузки, доступного через команду `journalctl` с опцией `-b` в первые 20 секунд с момента загрузки, начало загрузки определить автоматически. Для этого воспользуемся командой:

START=\$(journalctl -b --output=short-iso | head -n 1 | awk '{print \$1}')

END=\$(date -d "\$START + 20 seconds" +"%Y-%m-%d %H:%M:%S")

journalctl -b --since "\$START" --until "\$END" --no-pager

```
Mar 12 08:28:32 localhost dbus-daemon[687]: [system] Successfully activated service 'org.freedesktop.systemd1'
Mar 12 08:28:32 localhost polkitd[692]: Loading rules from directory /etc/polkit-1/rules.d
Mar 12 08:28:32 localhost polkitd[692]: Loading rules from directory /usr/share/polkit-1/rules.d
Mar 12 08:28:32 localhost systemd[1]: Starting systemd-hostnamed.service - Hostname Service...
Mar 12 08:28:32 localhost udisksd[701]: udisks daemon version 2.10.1 starting
Mar 12 08:28:32 localhost systemd[1]: Started rsyslog.service - System Logging Service.
Mar 12 08:28:32 localhost rsyslogd[729]: imuxsock: Acquired UNIX socket '/run/systemd/journal/syslog' (fd 3) from sys
d. [v8.2312.0]
Mar 12 08:28:32 localhost rsyslogd[729]: rsyslogd's groupid changed to 104
Mar 12 08:28:32 localhost rsyslogd[729]: rsyslogd's userid changed to 103
Mar 12 08:28:32 localhost rsyslogd[729]: [origin software="rsyslogd" swVersion="8.2312.0" x-pid="729" x-info="https://
w.rsyslog.com"] start
Mar 12 08:28:32 localhost systemd[1]: Finished sysstat.service - Resets System Activity Logs.
Mar 12 08:28:32 localhost dbus-daemon[687]: [system] Successfully activated service 'org.freedesktop.hostname1'
Mar 12 08:28:32 localhost systemd[1]: Started systemd-hostnamed.service - Hostname Service.
Mar 12 08:28:32 localhost dbus-daemon[687]: [system] Activating via systemd: service name='org.freedesktop.PolicyKit1
nit='polkit.service' requested by ':1.7' (uid=0 pid=731 comm="/usr/lib/systemd/systemd-hostnamed" label="unconfined")
Mar 12 08:28:32 localhost dbus-daemon[687]: [system] Successfully activated service 'org.freedesktop.PolicyKit1'
Mar 12 08:28:32 localhost systemd[1]: Started polkit.service - Authorization Manager.
Mar 12 08:28:32 localhost polkitd[692]: Finished loading, compiling and executing 6 rules
Mar 12 08:28:32 localhost polkitd[692]: Acquired the name org.freedesktop.PolicyKit1 on the system bus
Mar 12 08:28:32 localhost systemd[1]: Starting ModemManager.service - Modem Manager...
Mar 12 08:28:32 localhost ModemManager[748]: <msg> ModemManager (version 1.23.4) starting in system bus...
Mar 12 08:28:32 localhost systemd[1]: grub-common.service: Deactivated successfully.
Mar 12 08:28:32 localhost systemd[1]: Finished grub-common.service - Record successful boot for GRUB.
Mar 12 08:28:32 localhost systemd[1]: Starting grub-initrd-fallback.service - GRUB failed boot detection...
Mar 12 08:28:32 eltex-practice2-pg2-v11 systemd-hostnamed[731]: Hostname set to <eltex-practice2-pg2-v11> (transient)
```

Рисунок 1 – Записи из лога загрузки

Используя `awk` найдем все источники и их сообщения в файле `auth.log` (найдите его `find`), в названии источника удалите информацию об идентификаторе процесса при помощи `sed`, полученный результат отсортируйте по названию источника. Для этого воспользуемся командой: ***find /var/log -name "auth.log" 2>/dev/null***. Затем все источники и их информацию об идентификаторе процесса и удалим ее, используя команду: ***awk '{print \$5, \$0}' /var/log/auth.log | cut -d' ' -f1,6- | sed 's/[[0-9]*\]/g' | sort***


```

user1) user1) root on pts/0
user1) user1) root on pts/0
user1) user1) root on pts/0
user1) user1) root on pts/0
'user1' 'user1' to group 'Boychiki'
'user1' 'user1' to shadow group 'Boychiki'
user2) user2) root on pts/0
user2) user2) root on pts/0
user2) user2) root on pts/0
'user2' 'user2' to group 'Boychiki'
'user2' 'user2' to shadow group 'Boychiki'
user3) user3) root on pts/0
user3) user3) root on pts/0
user3) user3) root on pts/0
user3) user3) root on pts/0
user3) user3) root on pts/0
'user3' 'user3' to group 'Boychiki'
'user3' 'user3' to shadow group 'Boychiki'
user3) user3) user2 on pts/0
user: user: name=tech, UID=1000, GID=1000, home=/home/tech, shell=/bin/bash, from=none
user: user: name=user1, UID=1001, GID=1001, home=/home/user1, shell=/bin/bash, from=/dev/pts/0
user: user: name=user2, UID=1002, GID=1002, home=/home/user2, shell=/bin/bash, from=/dev/pts/0
user: user: name=user2, UID=1002, GID=1002, home=/home/user2, shell=/bin/sh, from=/dev/pts/0
user: user: name=user3, UID=1003, GID=1003, home=/home/user3, shell=/bin/bash, from=/dev/pts/0
user user 'user2'
user user 'user2' information
user user 'user3' information
user user 'user3' shell from '/bin/bash' to '/usr/sbin/nologin'
user user 'user3' shell from '/usr/sbin/nologin' to '/bin/bash'

```

Рисунок 2 – Удаление информации о id процесса

Для результата из предыдущего пункта найдем количество повторений для каждого источника и выведем их в виде списка «число повторений источник», результат отсортируйте по убыванию количества повторений. Для этого используем команду: **`awk '{print $5}' /var/log/auth.log | sed -E 's/[[0-9]]+|/|/g; s/:/|/g' | sort | uniq -c | sort -rn`**

```

792 session
38 rules
16 loading,
13 password
12 'tech'
10 user
10 the
10 system
9 listening
9 from
9 disconnect
8
7 group
6 user3)
6 signal
6 garbage
6 added
5 user1)
5 publickey
4 seat
4 login
3 user2)
3 is
2 'user3'
2 'user2'
2 'user1'
2 of

```

Рисунок 3 – Отсортированный список кол-во повторений

В файле `/etc/passwd` найдите всех пользователей в системе, у которых установлен shell `/usr/sbin/nologin` и выведите их в виде списка: «UID, username, список его групп», отсортированный в обратном порядке по UID, список групп используйте из файла `/etc/group`. Для этого воспользуемся командой: `awk -F: 'NR==FNR {groups[$3]=$1; next} $7==" /usr/sbin/nologin" {print $3 " ", " $1 ", " groups[$4]} /etc/group /etc/passwd | sort -t, -k1,1nr`

```
65534, nobody, nogroup
998, systemd-network, systemd-network
997, systemd-timesync, systemd-timesync
992, systemd-resolve, systemd-resolve
991, polkitd, polkitd
989, fwupd-refresh, fwupd-refresh
109, sshd, nogroup
108, usbmux, plugdev
107, landscape, landscape
105, tcpdump, tcpdump
104, uidd, uidd
103, syslog, syslog
101, messagebus, messagebus
42, _apt, nogroup
39, irc, irc
38, list, list
34, backup, backup
33, www-data, www-data
13, proxy, proxy
10, uucp, uucp
9, news, news
8, mail, mail
7, lp, lp
6, man, man
5, games, games
3, sys, sys
2, bin, bin
1, daemon, daemon
```

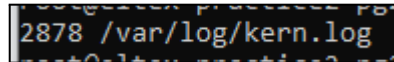
Рисунок 4 – Список пользователей

Найдите в результате вывода `dmesg` все строки, содержащие слово 'kernel'. Для этого воспользуемся командой: `dmesg | grep -i "kernel"`

```
0.000000] KERNEL supported cpus:
0.177735] Booting paravirtualized kernel on KVM
0.180904] Kernel command line: BOOT_IMAGE=/vmlinuz-6.8.0-55-generic root=/dev/mapper/ubuntu--vg-ubuntu--lv ro
0.181032] Unknown kernel command line parameters "BOOT_IMAGE=/vmlinuz-6.8.0-55-generic", will be passed to user space.
0.430484] Memory: 3933500K/4193760K available (22528K kernel code, 4443K rwdma, 14344K rodata, 4988K init, 4716K bss, 260000K reserved, 0K cma-reserved)
0.612829] DMA: preallocated 512 KiB GFP_KERNEL pool for atomic allocations
0.612997] DMA: preallocated 512 KiB GFP_KERNEL pool for atomic allocations
0.613167] DMA: preallocated 512 KiB GFP_KERNEL pool for atomic allocations
1.270918] Loaded X.509 cert 'Build time autogenerated kernel key: e0bedf6fa049a1e09bd65ebab3904f14cdd5bc68'
1.281942] Loaded X.509 cert 'Canonical Ltd. Kernel Module Signing: 88f752e560a1e0737e31163a466ad7b70a850c19'
1.406218] Loaded X.509 cert 'Build time autogenerated kernel key: e0bedf6fa049a1e09bd65ebab3904f14cdd5bc68'
1.456749] Freeing unused kernel image (initmem) memory: 4988K
1.457998] Write protecting the kernel read-only data: 38912k
1.459698] Freeing unused kernel image (rodata/data gap) memory: 2040K
9.646842] systemd[1]: Listening on systemd-udevd-kernel.socket - udev Kernel Socket.
9.729153] systemd[1]: Mounting sys-kernel-debug.mount - Kernel Debug File System...
9.734469] systemd[1]: Mounting sys-kernel-tracing.mount - Kernel Trace File System...
9.818018] systemd[1]: Starting modprobe@configfs.service - Load Kernel Module configfs...
9.827930] systemd[1]: Starting modprobe@dm_mod.service - Load Kernel Module dm_mod...
9.857365] systemd[1]: Starting modprobe@drm.service - Load Kernel Module drm...
9.964472] systemd[1]: Starting modprobe@efi_pstore.service - Load Kernel Module efi_pstore...
9.980636] systemd[1]: Starting modprobe@fuse.service - Load Kernel Module fuse...
9.991559] systemd[1]: Starting modprobe@loop.service - Load Kernel Module loop...
10.009206] systemd[1]: Starting systemd-modules-load.service - Load Kernel Modules...
10.024145] systemd[1]: Starting systemd-remount-fs.service - Remount Root and Kernel File Systems...
```

Рисунок 5 – Строки содержащие kernel

Подсчитаем количество строк в файле /var/log/kern.log. Для этого воспользуемся командой: ***wc -l /var/log/kern.log***



```
2878 /var/log/kern.log
```

Рисунок 6 – Кол-во строк в файле /var/log/kern.log

Отформатируем вывод записей в /var/log/apt/history.log в следующем порядке, построчно: Commandline: ... ; Start-Date: ... ; End-Date:. Для этого воспользуемся командой: ***sudo awk -v RS="" -v OFS='; ' ' '***

```
{
    cmd=""; start=""; end="";
    for(i=1; i<=NF; i++) {
        if ($i == "Commandline:") {
            for(j=i+1; j<=NF && $j !~ /\:/; j++) cmd = (cmd=="" ? "" : cmd " ") $j
        }
        if ($i == "Start-Date:") start = $(i+1) " " $(i+2);
        if ($i == "End-Date:") end = $(i+1) " " $(i+2);
    }
    if (cmd != "") print "Commandline: " cmd, "Start-Date: " start, "End-Date: " end
}' /var/log/apt/history.log
```

```
Commandline: /usr/bin/unattended-upgrade ; Start-Date: 2026-03-13 08:10:22 ; End-Date: 2026-03-13 08:14:21
Commandline: /usr/bin/unattended-upgrade ; Start-Date: 2026-03-13 08:14:31 ; End-Date: 2026-03-13 08:15:15
Commandline: /usr/bin/unattended-upgrade ; Start-Date: 2026-03-13 08:15:20 ; End-Date: 2026-03-13 08:17:27
Commandline: /usr/bin/unattended-upgrade ; Start-Date: 2026-03-13 08:17:32 ; End-Date: 2026-03-13 08:18:23
Commandline: /usr/bin/unattended-upgrade ; Start-Date: 2026-03-13 08:18:35 ; End-Date: 2026-03-13 08:21:33
Commandline: /usr/bin/unattended-upgrade ; Start-Date: 2026-03-13 08:21:44 ; End-Date: 2026-03-13 08:22:47
Commandline: /usr/bin/unattended-upgrade ; Start-Date: 2026-03-13 08:23:02 ; End-Date: 2026-03-13 08:26:39
Commandline: /usr/bin/unattended-upgrade ; Start-Date: 2026-03-13 08:26:46 ; End-Date: 2026-03-13 08:28:36
Commandline: /usr/bin/unattended-upgrade ; Start-Date: 2026-03-13 08:28:41 ; End-Date: 2026-03-13 08:29:29
Commandline: /usr/bin/unattended-upgrade ; Start-Date: 2026-03-13 08:29:33 ; End-Date: 2026-03-13 08:32:31
Commandline: /usr/bin/unattended-upgrade ; Start-Date: 2026-03-13 08:32:46 ; End-Date: 2026-03-13 08:38:24
Commandline: /usr/bin/unattended-upgrade ; Start-Date: 2026-03-13 08:38:42 ; End-Date: 2026-03-13 08:42:08
Commandline: /usr/bin/unattended-upgrade ; Start-Date: 2026-03-13 08:42:12 ; End-Date: 2026-03-13 08:42:50
Commandline: /usr/bin/unattended-upgrade ; Start-Date: 2026-03-13 08:42:58 ; End-Date: 2026-03-13 08:43:46
Commandline: /usr/bin/unattended-upgrade ; Start-Date: 2026-03-13 08:43:51 ; End-Date: 2026-03-13 08:44:57
Commandline: /usr/bin/unattended-upgrade ; Start-Date: 2026-03-13 08:45:01 ; End-Date: 2026-03-13 08:45:57
Commandline: /usr/bin/unattended-upgrade ; Start-Date: 2026-03-13 08:46:06 ; End-Date: 2026-03-13 08:46:56
Commandline: /usr/bin/unattended-upgrade ; Start-Date: 2026-03-13 08:47:03 ; End-Date: 2026-03-13 08:48:32
Commandline: /usr/bin/unattended-upgrade ; Start-Date: 2026-03-13 08:48:37 ; End-Date: 2026-03-13 08:50:16
Commandline: /usr/bin/unattended-upgrade ; Start-Date: 2026-03-13 08:50:19 ; End-Date: 2026-03-13 08:54:33
Commandline: /usr/bin/unattended-upgrade ; Start-Date: 2026-03-13 08:54:37 ; End-Date: 2026-03-13 08:58:14
Commandline: /usr/bin/unattended-upgrade ; Start-Date: 2026-03-13 08:58:21 ; End-Date: 2026-03-13 09:00:43
Commandline: /usr/bin/unattended-upgrade ; Start-Date: 2026-03-13 09:00:48 ; End-Date: 2026-03-13 09:01:55
Commandline: /usr/bin/unattended-upgrade ; Start-Date: 2026-03-13 09:01:59 ; End-Date: 2026-03-13 09:03:53
Commandline: /usr/bin/unattended-upgrade ; Start-Date: 2026-03-13 09:03:59 ; End-Date: 2026-03-13 09:04:31
Commandline: /usr/bin/unattended-upgrade ; Start-Date: 2026-03-13 09:04:35 ; End-Date: 2026-03-13 09:06:39
```

Рисунок 7 – Отформатированный вывод записей

Выполнение лабораторной работы №2. Раздел № 3

Для начала установим *git*. Для этого используем команду: ***apt install -y git gh***

```
WARNING: apt does not have a stable CLI interface. Use with caution in scripts.

Reading package lists...
Building dependency tree...
Reading state information...
git is already the newest version (1:2.43.0-1ubuntu7.3).
git set to manually installed.
The following NEW packages will be installed:
  gh
0 upgraded, 1 newly installed, 0 to remove and 87 not upgraded.
Need to get 8,836 kB of archives.
After this operation, 45.4 MB of additional disk space will be used.
Get:1 http://ru.archive.ubuntu.com/ubuntu noble-updates/universe amd64 gh amd64 2.45.0-1ubuntu0.3 [8,836 kB]
Fetched 8,836 kB in 1s (9,897 kB/s)

Selecting previously unselected package gh.
(Reading database ... 125247 files and directories currently installed.)
Preparing to unpack .../gh_2.45.0-1ubuntu0.3_amd64.deb ...
Unpacking gh (2.45.0-1ubuntu0.3) ...
Setting up gh (2.45.0-1ubuntu0.3) ...
Processing triggers for man-db (2.12.0-4build2) ...
Scanning processes...
Scanning candidates...
Scanning linux images...
```

Рисунок 1 – Установка *git*

Установим параметры *git* в соответствии с нашими учетными данными. Для этого воспользуемся командами:

```
git config --global user.name "имя пользователя github"
```

```
git config --global user.email "ваш e-mail, использованный при регистрации"
```

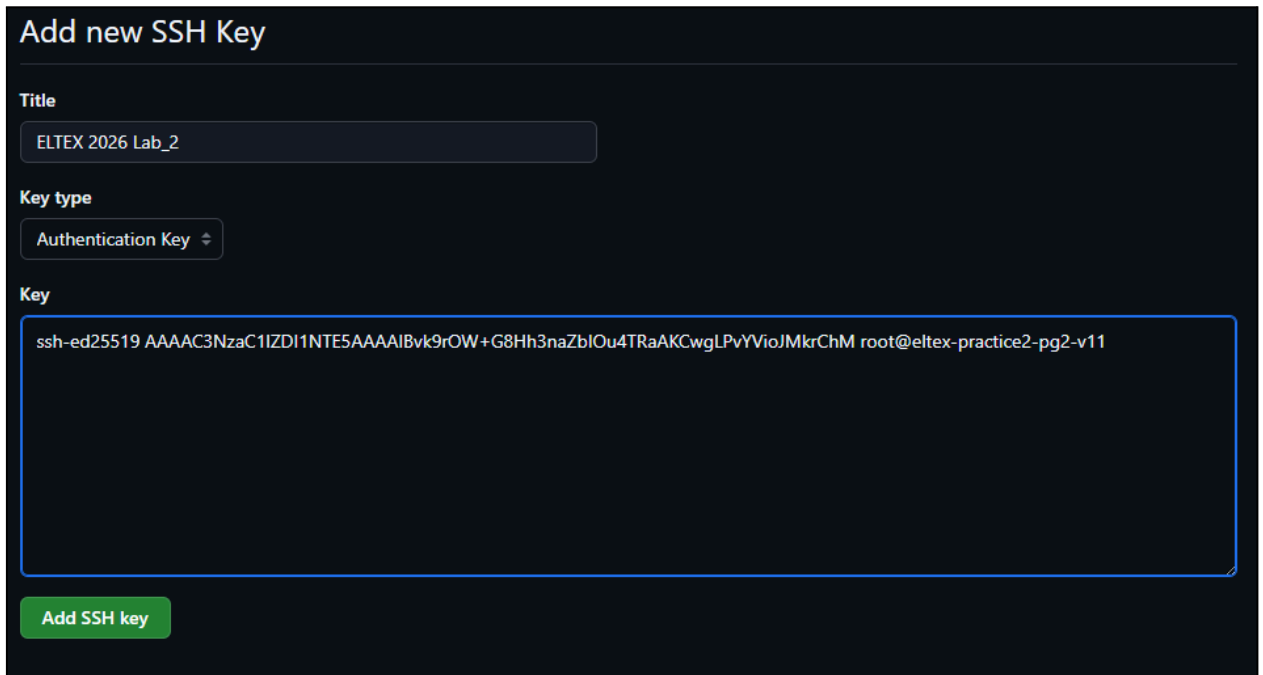
```
git config --global init.defaultBranch main
```

Далее создадим *ssh* ключ. Для этого воспользуемся командой: ***ssh-keygen -t ed25519 -f ~/.ssh/SolomonDi***

```
Your identification has been saved in /root/.ssh/SolomonDi
Your public key has been saved in /root/.ssh/SolomonDi.pub
The key fingerprint is:
SHA256:rZY1W5UKUE6UxJ1RIpYmWnpkowLtf50RJbHjnmf2B1M root@eltex-practice2-pg2-v11
The key's randomart image is:
---[ED25519 256]---+
.   o*B+o+. |
. .   B**o. . |
o   BoBo   o |
o +.+o . oE |
o .So+ o. |
. .+= +o |
. *. = o |
. + . . |
. . |
-----[SHA256]-----+
```

Рисунок 2 – Генерация *ssh* ключа

Далее добавим ssh ключ. Для этого зайдём на github и добавим ssh ключ.



Add new SSH Key

Title
ELTEX 2026 Lab_2

Key type
Authentication Key

Key
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIBvk9rOW+G8Hh3naZbIOu4TRaAKCwgLPvYVioJMkrChM root@eltex-practice2-pg2-v11

Add SSH key

Рисунок 3 – Добавление ssh ключа

Подключите этот компьютер к github. Используя команду: *gh auth login*

```
root@eltex-practice2-pg2-v11:~# echo "> gh auth login" >> practice2_${USER}_part3.log
root@eltex-practice2-pg2-v11:~# gh auth login
? What account do you want to log into? GitHub.com
? What is your preferred protocol for Git operations on this host? SSH
? Upload your SSH public key to your GitHub account? /root/.ssh/SolomonDi.pub
? Title for your SSH key: eltex-practice2-pg2-v11
? How would you like to authenticate GitHub CLI? Login with a web browser

! First copy your one-time code: 406D-3ABE
Press Enter to open github.com in your browser...
```

Рисунок 4 – Токен

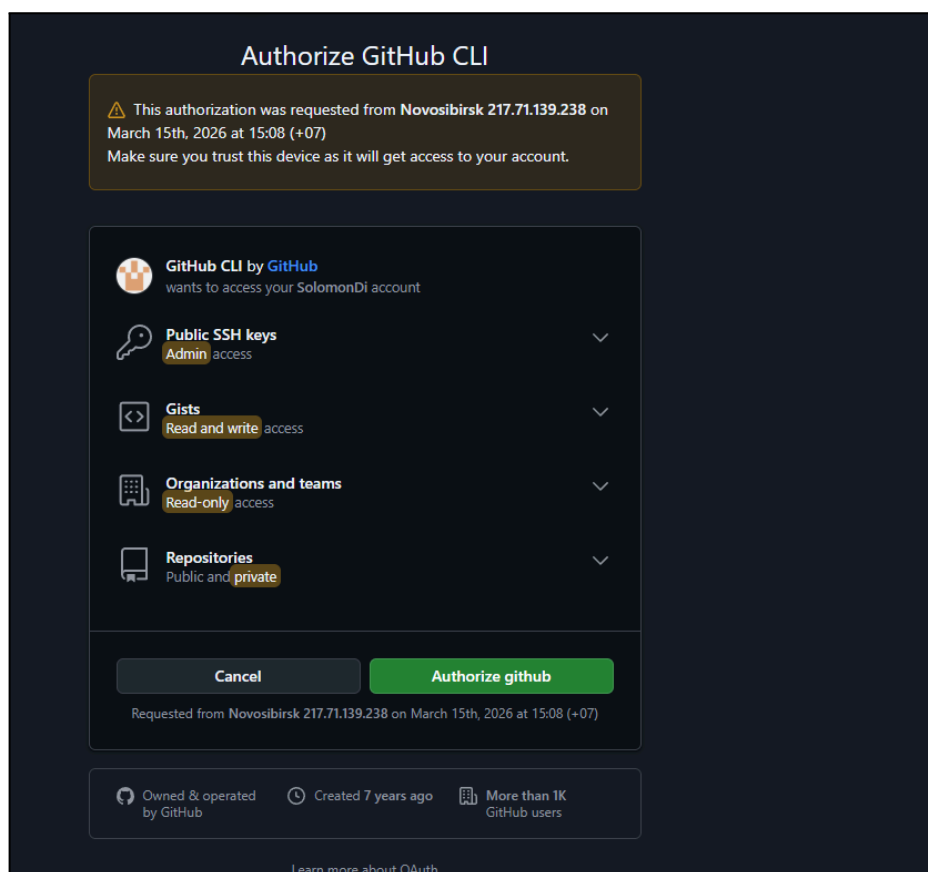


Рисунок 5 – Авторизация на github

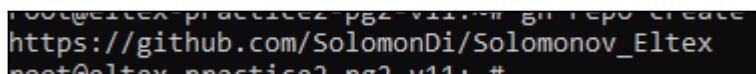
Укажем автоматическую авторизацию только для сайта github.com. Используя команду: ***gh auth setup-git --hostname github.com***

Проверим, что все в порядке. Для этого используем команду: ***gh auth status***

```
github.com
  Logged in to github.com account SolomonDi (/root/.config/gh/hosts.yml)
  - Active account: true
  - Git operations protocol: ssh
  - Token: gho_*****
  - Token scopes: 'admin:public_key', 'gist', 'read:org', 'repo'
```

Рисунок 6 – git status

Создадим репозиторий на сервере github. Для этого используем команду: ***gh repo create --public Solomonov_Eltex***



```
root@eltex-practice2-pg2-v11:~# gh repo create
https://github.com/SolomonDi/Solomonov_Eltex
root@eltex-practice2-pg2-v11:~#
```

Рисунок 7 – Репозиторий создан

Далее все команды и их вывод находятся в log file. в module1_lab_2/task3